

ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH

**TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN**



PHÂN TÍCH DỮ LIỆU

BÁO CÁO

Sinh viên thực hiện:

Đỗ Hữu Lộc - 3123410201

Nguyễn Hoàng Phúc - 3123410277

Hồ Đắc Khả - 31234110143

Giảng viên: **Đỗ Như Tài**

Thành phố Hồ Chí Minh, 10 tháng 10 năm 2025

Mục lục

Data Analysis Examples.....	3
I. Bitly Data from 1.USA.gov.....	3
1. Giới thiệu.....	3
2. Mục tiêu.....	4
3. Tổng quan dữ liệu.....	4
4. Phân tích múi giờ với Pandas.....	5
4.1. Phân bố múi giờ.....	5
4.2. Phân bố múi giờ theo hệ điều hành.....	6
5. Kết luận.....	8
II. MovieLens 1M Dataset.....	8
1. Giới thiệu.....	8
2. Mục tiêu.....	9
3. Tổng quan dữ liệu.....	9
4. Phân tích dữ liệu.....	10
4.1. Xử lý dữ liệu.....	10
4.2. Top phim được nữ giới đánh giá cao nhất.....	10
4.3. So sánh với sự khác biệt giới tính trong đánh giá phim.....	11
4.4. Phân bố thể loại phim.....	12
5. Kết luận.....	12
III. US Baby Names 1880–2010.....	13
1. Giới thiệu.....	13
2. Mục tiêu.....	13
3. Tổng quan dữ liệu.....	13
4. Phân tích dữ liệu.....	14
4.1. Gộp tất cả các năm (1880-2010) vào một DataFrame.....	14
4.2. Phân tích theo tổng số sinh theo giới tính và năm sinh.....	14
4.3. Phân tích xu hướng một vài tên phổ biến.....	15
4.4. Phân tích sự đa dạng của tên.....	16
4.5. Tỷ lệ bé trai có tên kết thúc bằng d/n/y theo thời gian.....	17
5. Kết luận.....	18
IV. Cơ sở dữ liệu thực phẩm USDA.....	19
1. Giới thiệu.....	19
2. Mục tiêu.....	19
3. Tổng quan dữ liệu.....	19
3.1. Chuẩn bị dữ liệu.....	19
3.2. Tiền xử lý dữ liệu.....	20
3.2.1 Trích xuất thông tin.....	20
3.2.2 Gộp dữ liệu.....	21
3.2.3 Làm sạch dữ liệu.....	22
3.3 Phân tích và Trực quan hóa dữ liệu.....	22
3.3.1 Hàm lượng Dinh dưỡng Trung bình theo Nhóm Thực phẩm.....	22
3.3.2 Xác định các Thực phẩm Giàu Dinh dưỡng Hàng đầu.....	23

3.3.3	Mối tương quan giữa các Chất dinh dưỡng.....	24
4.	Kết luận.....	24
V.	Phân tích Đóng góp cho Cuộc Bầu cử Tổng thống Hoa Kỳ năm 2012.....	25
1.	Giới thiệu.....	25
2.	Mục tiêu.....	25
3.	Tổng quan dữ liệu.....	25
3.1	Chuẩn bị dữ liệu.....	25
3.2	Tiền xử lí dữ liệu.....	26
3.2.1.	Xử lý trùng lặp.....	26
3.2.2.	Lọc dữ liệu.....	26
3.2.3	Thêm thông tin Đảng phái.....	26
3.4	Phân tích và trực quan hóa dữ liệu.....	27
3.4.1	Phân tích Tổng đóng góp theo Đảng phái.....	27
3.4.2	Phân tích Đóng góp theo Tiểu bang.....	28
4.	Kết Luận.....	28
	Phụ lục A : Numpy Nâng cao.....	30
A.1.	Cấu trúc nội tại của đối tượng ndarray (ndarray Object Internals).....	30
A.2.	Thao tác Mảng Nâng cao (Advanced Array Manipulation).....	33
A.3.	Broadcasting.....	43
A.4.	Sử dụng ufunc Nâng cao (Advanced ufunc Usage).....	47
A.5.	Mảng có cấu trúc và Mảng bản ghi (Structured and Record Arrays).....	47
A.6.	Sắp xếp (More About Sorting).....	49
A.7.	Viết hàm NumPy nhanh bằng Numba (Writing Fast NumPy Functions with Numba).....	49
A.8.	Thao tác I/O Nâng cao trên Mảng (Advanced Array Input and Output).....	50
A.9.	Mẹo về Hiệu suất (Performance Tips).....	50

MSSV	Họ tên	Phân công công việc	Mức độ hoàn thành
Hồ Đắc Khả	31234110143	Phần 13.4-13.6	100%
Đỗ Hữu Lộc	3123410201		100%
Nguyễn Hoàng Phúc	3123410277	Phụ lục A : Numpy Nâng cao	100%

Data Analysis Examples

I. Bitly Data from 1.USA.gov

1. Giới thiệu

Bộ dữ liệu Bitly Data from 1.USA.gov bao gồm các bản ghi truy cập các đường liên kết rút gọn được tạo thông qua tên miền chính thức 1.usa.gov. Dữ liệu này được thu thập từ dịch vụ rút gọn liên kết Bitly, phản ánh mức độ quan tâm của người dùng Internet đối với các trang web và tài nguyên của chính phủ Hoa Kỳ. Mỗi bản ghi chứa thông tin như múi giờ, hệ điều hành, nguồn truy cập và thời gian nhấp chuột. Bộ dữ liệu này thường được sử dụng trong các bài học phân tích dữ liệu để thực hành làm sạch, trích xuất và trực quan hóa dữ liệu từ nguồn thực tế.

Bộ dữ liệu tham khảo từ pydata-book/datasets/bitly_usagov at 3rd-edition · wesm/pydata-book.

2. Mục tiêu

Mục tiêu của việc phân tích bộ dữ liệu Bitly Data from 1.USA.gov là nhằm hiểu rõ hành vi truy cập của người dùng Internet đối với các liên kết rút gọn thuộc miền chính phủ Hoa Kỳ. Cụ thể, quá trình phân tích tập trung vào:

Xác định phân bố múi giờ và thời điểm người dùng truy cập nhiều nhất.

Phân tích thiết bị và hệ điều hành được sử dụng phổ biến khi truy cập các đường dẫn 1.usa.gov.

Đánh giá nguồn lưu lượng truy cập (referrer) và xác định mức độ tương tác theo khu vực địa lý.

Minh họa khả năng ứng dụng xử lý dữ liệu JSON, làm sạch dữ liệu và trực quan hóa hành vi người dùng trong phân tích dữ liệu thực tế.

3. Tổng quan dữ liệu

Bộ dữ liệu Bitly Data from 1.USA.gov được thu thập từ các liên kết rút gọn thuộc miền chính phủ Hoa Kỳ. Dữ liệu thường ở dạng JSON, trong đó mỗi bản ghi đại diện cho một lần nhấp chuột (click) vào liên kết.

#	Column	Non-Null Count	Dtype
0	a	3440 non-null	object
1	c	2919 non-null	object
2	nk	3440 non-null	float64
3	tz	3440 non-null	object
4	gr	2919 non-null	object
5	g	3440 non-null	object
6	h	3440 non-null	object
7	l	3440 non-null	object
8	al	3094 non-null	object
9	hh	3440 non-null	object
10	r	3440 non-null	object
11	u	3440 non-null	object
12	t	3440 non-null	float64
13	hc	3440 non-null	float64
14	cy	2919 non-null	object
15	ll	2919 non-null	object
16	_heartbeat_	120 non-null	float64
17	kw	93 non-null	object

dtypes: float64(4), object(14)
memory usage: 500.8+ KB

Cấu trúc và ý nghĩa các biến chính:

a: chuỗi mô tả thông tin trình duyệt và hệ điều hành của người dùng (user agent).

c: mã quốc gia (country code) của người dùng theo chuẩn ISO, ví dụ như “US”, “CA”, “GB”.

nk: giá trị nhị phân thể hiện người dùng mới hay đã từng truy cập (0 hoặc 1).

tz: múi giờ của người dùng, chẳng hạn “America/New_York”.

gr: mã bang hoặc khu vực (region) của người dùng ở Hoa Kỳ.

g: mã định danh của liên kết rút gọn (short link ID).

u: đường dẫn URL đầy đủ mà người dùng được chuyển đến.

r: địa chỉ trang web giới thiệu (referrer), chỉ nguồn mà người dùng đến từ đó.

t: thời điểm truy cập (timestamp) được lưu theo định dạng Unix time.

Bộ dữ liệu này thường có kích thước vài chục nghìn bản ghi và được dùng trong các bài thực hành

về làm sạch dữ liệu JSON, phân tích hành vi truy cập cũng như trực quan hóa thời gian truy cập theo múi giờ hoặc khu vực.

4. Phân tích múi giờ với Pandas

4.1. Phân bố múi giờ

Phần này mô tả cách sử dụng pandas và seaborn để xử lý, phân tích và trực quan hóa dữ liệu múi giờ từ tập dữ liệu `records`. Các bước dưới đây giúp chuyển từ xử lý thủ công sang dùng công cụ chuyên biệt trong Python để phân tích dữ liệu lớn dễ dàng hơn.

*Cột tz (múi giờ):

```
frame["tz"].head()

0    America/New_York
1    America/Denver
2    America/New_York
3    America/Sao_Paulo
4    America/New_York
Name: tz, dtype: object
```

* Làm sạch dữ liệu múi giờ:

Thay thế NaN → “Missing”

Chuỗi rỗng → “Unknown”

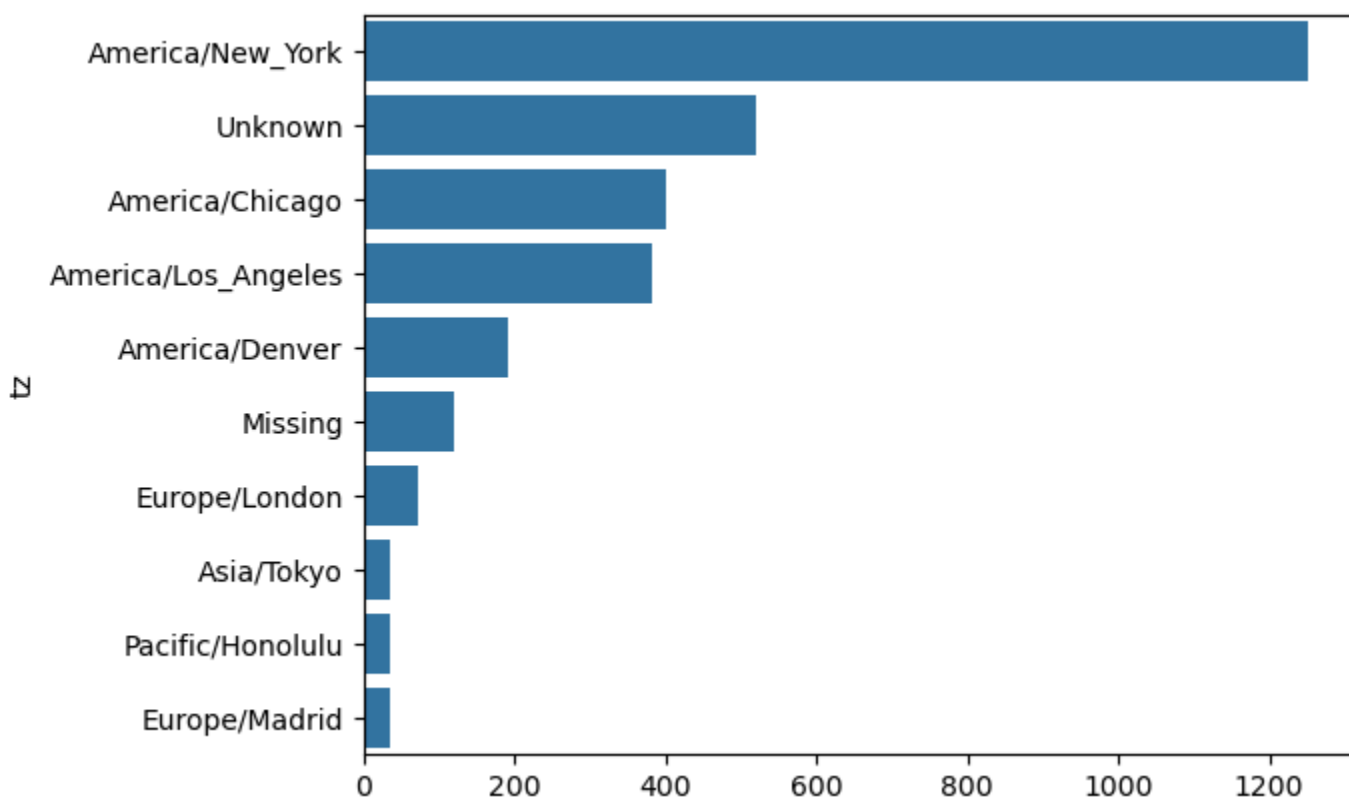
```
clean_tz = frame["tz"].fillna("Missing")
clean_tz[clean_tz == ""] = "Unknown"
tz_counts = clean_tz.value_counts()
tz_counts.head()
```

Kết quả thực hiện:

```
tz
America/New_York    1251
Unknown              521
America/Chicago     400
America/Los_Angeles 382
America/Denver      191
Name: count, dtype: int64
```

*Biểu đồ thanh ngang với seaborn:

Phân tích múi giờ của người dùng cho thấy America/New_York là múi giờ phổ biến nhất, chiếm một phần đáng kể trong tổng số lượt truy cập. Ngoài ra, có một lượng lớn các bản ghi không xác định múi giờ (Unknown). Điều này có thể do dữ liệu bị thiếu hoặc không thể xác định được múi giờ từ thông tin tác nhân người dùng.



Hình 13.1.1 Biểu đồ thanh ngang (seaborn) phân bố top 10 múi giờ phổ biến nhất.

Nhận xét:

America/New_York có số lượng truy cập cao nhất, cho thấy một lượng lớn người dùng đến từ khu vực này.

Các múi giờ Unknown và Missing chiếm vị trí thứ hai và thứ sáu, điều này cần được xem xét kỹ hơn trong các phân tích tiếp theo để hiểu rõ nguyên nhân của dữ liệu thiếu.

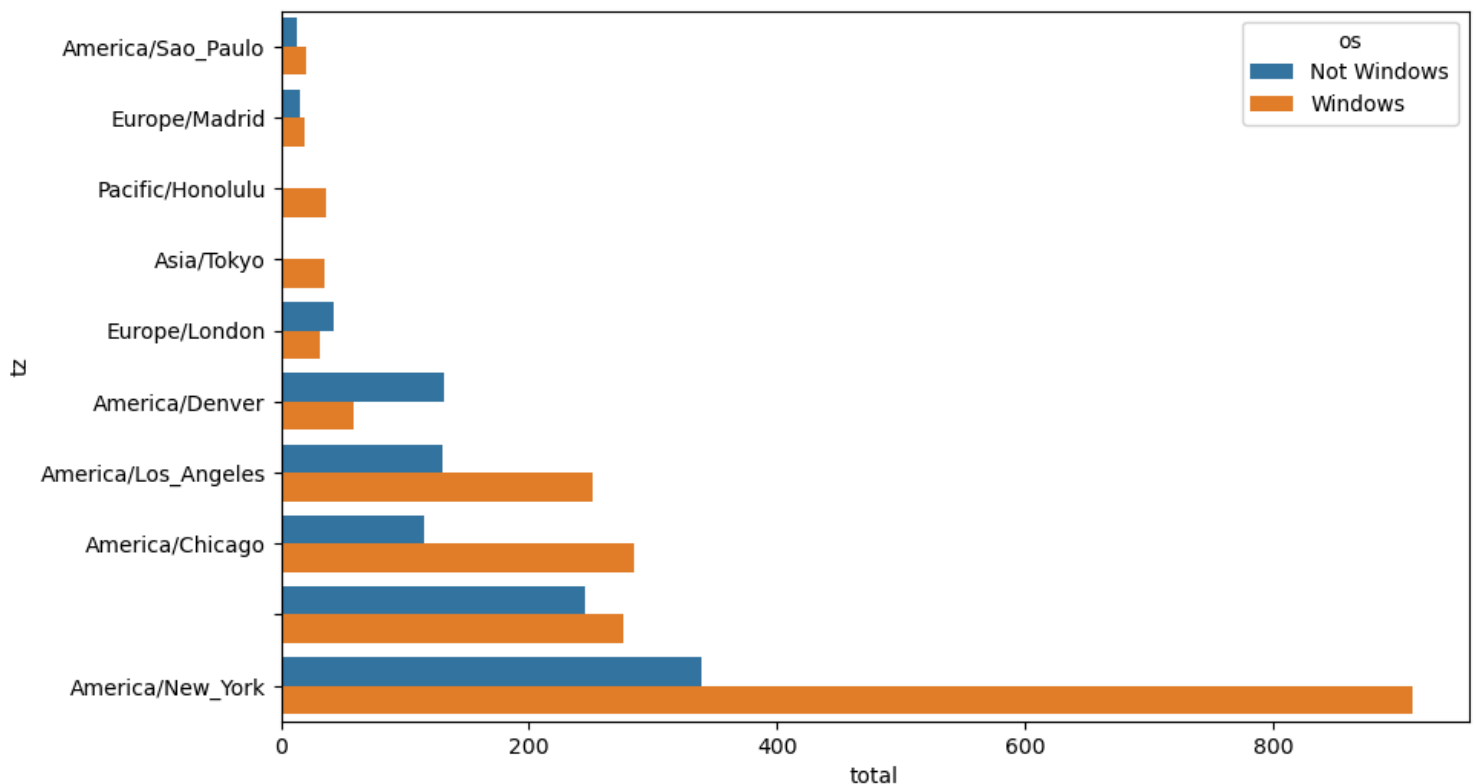
Các múi giờ khác như America/Chicago, America/Los_Angeles, America/Denver cũng có số lượng đáng kể, phản ánh sự phân bố người dùng trên khắp nước Mỹ và một số khu vực quốc tế.

4.2. Phân bố múi giờ theo hệ điều hành

Phân tích này xem xét sự phân bố hệ điều hành (Windows và phần còn lại) trong các múi giờ hàng đầu. Dữ liệu tác nhân người dùng (a) được sử dụng để suy ra hệ điều hành của người dùng.

os	Not Windows	Windows
tz		
America/Sao_Paulo	13.0	20.0
Europe/Madrid	16.0	19.0
Pacific/Honolulu	0.0	36.0
Asia/Tokyo	2.0	35.0
Europe/London	43.0	31.0
America/Denver	132.0	59.0
America/Los_Angeles	130.0	252.0
America/Chicago	115.0	285.0
	245.0	276.0
America/New_York	339.0	912.0

Hình 13.1.2 Bảng top 10 múi giờ phổ biến nhất theo hệ điều hành.



Hình 13.1.3. Biểu đồ hiển thị top 10 số lượng người dùng Window và không Window theo múi giờ.
Nhận xét:

Hệ điều hành Windows chiếm ưu thế đáng kể trong hầu hết các múi giờ, đặc biệt là ở America/New_York, America/Chicago và America/Los_Angeles.

Múi giờ America/Denver có một lượng lớn người dùng được phân loại là không Window, điều này có thể chỉ ra sự đa dạng về thiết bị hoặc tác nhân người dùng không được phân loại rõ

ràng.

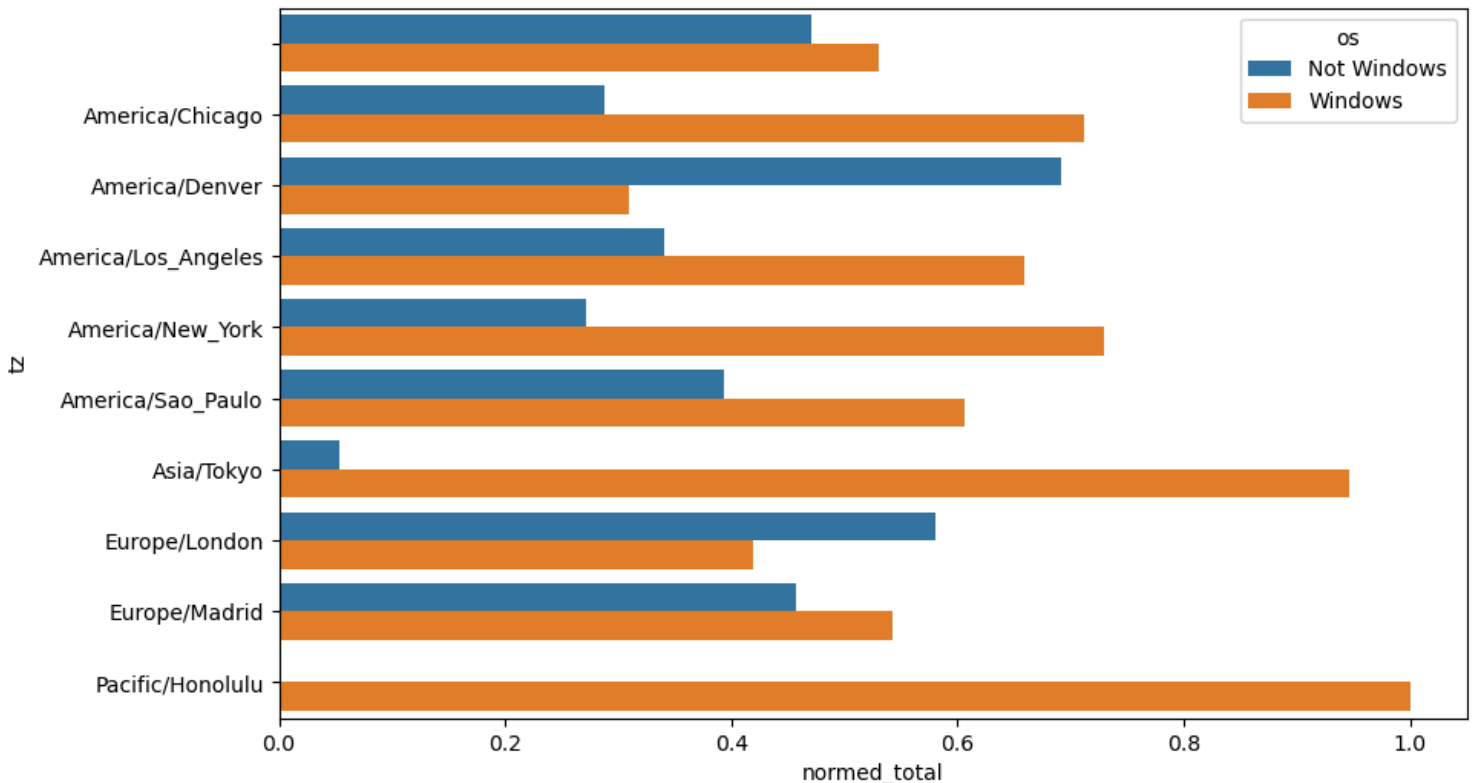
Các múi giờ quốc tế như Asia/Tokyo, Europe/London, Europe/Madrid có số lượng người dùng thấp hơn và chủ yếu là Windows.

* Chuẩn hóa tỉ lệ phần trăm để so sánh:

Thay vì dùng số lượng tuyệt đối, ta tính tỉ lệ phần trăm trong từng múi giờ.

```
def norm_total(group):  
    group["normed_total"] = group["total"] / group["total"].sum()  
    return group  
  
results = count_subset.groupby("tz").apply(norm_total)
```

Biểu đồ dạng chuẩn hóa phần trăm



Hình 13.1.4 Biểu đồ thanh top 10 phần trăm tỉ lệ người dùng hệ điều hành.

Nhận xét:

Ngoại trừ múi giờ “America/Denver” có xu hướng dùng hệ điều hành non-Window thì các múi giờ còn lại có xu thế dùng Window.

Đặc biệt “Pacific/Honolulu” có toàn bộ người dùng là Window.

5. Kết luận

Phân tích dữ liệu Bitly đã cung cấp cái nhìn sâu sắc về phân bố địa lý và hệ điều hành của người dùng truy cập các liên kết của USA.gov, với America/New_York và hệ điều hành Windows chiếm ưu thế.

Điều này giúp xác định xu hướng và độ ưa thích của người dùng theo múi giờ đối với các hệ điều hành. Xác định xu hướng thị trường.

II. MovieLens 1M Dataset

1. Giới thiệu

Bộ dữ liệu MovieLens 1M được phát hành bởi GroupLens Research thuộc Đại học Minnesota,

chứa hơn 1 triệu đánh giá phim do khoảng 6.000 người dùng thực hiện trên hơn 3.900 bộ phim. Mỗi đánh giá được gắn với thông tin về người dùng (tuổi, giới tính, nghề nghiệp) và thông tin phim (tên, thể loại). Đây là một trong những tập dữ liệu kinh điển trong lĩnh vực hệ thống gợi ý (Recommendation Systems), được dùng để nghiên cứu và thực hành các thuật toán lọc cộng tác (Collaborative Filtering), phân tích sở thích và xây dựng mô hình đề xuất phim cá nhân hóa.

Bộ dữ liệu tham khảo từ [pydata-book/datasets/movielens at 3rd-edition](https://pydata-book/datasets/movielens-at-3rd-edition) · wesm/pydata-book.

2. Mục tiêu

Mục tiêu của việc phân tích bộ dữ liệu MovieLens 1M là khai thác hành vi và sở thích của người xem phim, từ đó phục vụ việc xây dựng và đánh giá mô hình gợi ý phim cá nhân hóa. Các mục tiêu cụ thể gồm:

- Phân tích phân bố điểm đánh giá theo người dùng và thể loại phim.

- Khám phá mối quan hệ giữa đặc điểm người dùng (tuổi, giới tính, nghề nghiệp) với xu hướng đánh giá phim.

- Xác định các bộ phim phổ biến hoặc gây tranh cãi nhất dựa trên mức độ và độ phân tán của đánh giá.

- Minh họa việc áp dụng kỹ thuật lọc cộng tác (Collaborative Filtering) hoặc phân tích tương quan người dùng trong khai phá dữ liệu.

3. Tổng quan dữ liệu

Bộ dữ liệu MovieLens 1M được phát hành bởi GroupLens Research của Đại học Minnesota, bao gồm hơn một triệu lượt đánh giá phim từ hơn sáu nghìn người dùng cho gần bốn nghìn bộ phim. Dữ liệu được chia thành ba tệp chính là users.dat, movies.dat và ratings.dat.

Trong tệp users.dat, các biến mô tả thông tin của người dùng bao gồm mã người dùng (UserID), giới tính (Gender), nhóm tuổi (Age), nghề nghiệp (Occupation) và mã vùng bưu điện (Zip-code). Các biến này giúp xác định đặc điểm nhân khẩu học của người dùng, phục vụ cho việc phân tích xu hướng đánh giá theo giới tính, độ tuổi hoặc nghề nghiệp.

Tệp movies.dat chứa thông tin về các bộ phim, bao gồm mã phim (MovieID), tên phim kèm năm phát hành (Title) và danh sách thể loại (Genres). Thông tin này cho phép nghiên cứu phân bố đánh giá theo thể loại hoặc xác định những bộ phim được yêu thích nhất trong từng nhóm khán giả.

Tệp ratings.dat là dữ liệu trung tâm, lưu trữ các đánh giá mà người dùng gán cho từng bộ phim. Mỗi dòng bao gồm mã người dùng, mã phim, điểm đánh giá (thang điểm từ 1 đến 5) và thời điểm đánh giá (Timestamp). Bộ dữ liệu này thường được dùng để huấn luyện và kiểm thử các mô hình gợi ý phim dựa trên hành vi người dùng.

Bộ dữ liệu bao gồm: ~1.000.000 lượt đánh giá, ~6.000 người dùng, ~4.000 phim.

	user_id	movie_id	rating	timestamp
	0	1	1193	5
	1	1	661	3
	2	1	914	3
	3	1	3408	4
	4	1	2355	5
	...	...	...	...
	1000204	6040	1091	1
	1000205	6040	1094	5
	1000206	6040	562	5
	1000207	6040	1096	4
	1000208	6040	1097	4

1000209 rows × 4 columns

4. Phân tích dữ liệu

4.1. Xử lý dữ liệu

Gộp 3 bảng thành 1 bảng tổng hợp: Dễ dàng phân tích khi các thông tin của phim, người dùng, và điểm đánh giá nằm cùng một dòng.

```
data = pd.merge(pd.merge(ratings, users), movies)
data
data.iloc[0]
```

Kết quả thực hiện:

```
user_id          1
movie_id        1193
rating           5
timestamp       978300760
gender           F
age             1
occupation      10
zip             48067
title           One Flew Over the Cuckoo's Nest (1975)
genres          Drama
Name: 0, dtype: object
```

4.2. Top phim được nữ giới đánh giá cao nhất

Các bộ phim được nữ giới đánh giá cao nhất thường có điểm trung bình cao hơn từ phía họ.

```
top_female_ratings = mean_ratings.sort_values("F", ascending=False)
top_female_ratings.head()
```

Kết quả thực hiện:

gender	F	M
title		
Close Shave, A (1995)	4.644444	4.473795
Wrong Trousers, The (1993)	4.588235	4.478261
Sunset Blvd. (a.k.a. Sunset Boulevard) (1950)	4.572650	4.464589
Wallace & Gromit: The Best of Aardman Animation (1996)	4.563107	4.385075
Schindler's List (1993)	4.562602	4.491415

Hình 13.2.1 Top 5 bộ phim được nữ giới đánh giá cao nhất.

Nhận xét:

Trung bình, nữ giới có xu hướng chấm điểm cao hơn nam giới đối với các phim có nội dung nhân văn, cảm động hoặc mang yếu tố hoạt hình, nghệ thuật.

Sự chênh lệch tuy không quá lớn (khoảng 0.1–0.2 điểm), nhưng ổn định ở tất cả phim $\Rightarrow$ cho thấy một khuynh hướng thị hiếu nhất quán.

Những kết quả này có thể được khai thác để xây dựng mô hình gợi ý phim phù hợp hơn cho người dùng nữ, ưu tiên các thể loại hoạt hình, tâm lý – tình cảm và phim cổ điển có giá trị nghệ thuật.

4.3. So sánh với sự khác biệt giới tính trong đánh giá phim

Mục đích chính của việc so sánh độ ưa thích phim theo giới tính là nhận diện sự khác biệt trong thị hiếu, cải thiện đề xuất cá nhân hóa, hỗ trợ chiến lược nội dung – marketing, và hiểu rõ hơn về xu hướng văn hóa – xã hội của người xem.

Để hiểu rõ hơn về sự khác biệt trong sở thích, chúng ta tính toán sự chênh lệch điểm trung bình giữa nam và nữ (Điểm Nam - Điểm Nữ) ($\text{mean_rating}["M"] - \text{mean_ratings}["F"]$).

Sắp xếp tăng dần $\rightarrow$ phim phụ nữ thích hơn đàn ông.

gender	F	M	diff
title			
Dirty Dancing (1987)	3.790378	2.959596	-0.830782
Jumpin' Jack Flash (1986)	3.254717	2.578358	-0.676359
Grease (1978)	3.975265	3.367041	-0.608224
Little Women (1994)	3.870588	3.321739	-0.548849
Steel Magnolias (1989)	3.901734	3.365957	-0.535777

Hình 13.2.2 Top 5 bộ phim nữ giới thích hơn nam giới

Nhận xét:

Các bộ phim trong danh sách này có xu hướng được nữ giới đánh giá cao hơn đáng kể so với nam giới. Đây thường là các bộ phim thuộc thể loại lãng mạn, nhạc kịch hoặc chính kịch tập trung vào các mối quan hệ và cảm xúc.

Sắp xếp tăng dần $\rightarrow$ phim nam giới thích hơn nữ giới.

gender	F	M	diff
title			
Good, The Bad and The Ugly, The (1966)	3.494949	4.221300	0.726351
Kentucky Fried Movie, The (1977)	2.878788	3.555147	0.676359
Dumb & Dumber (1994)	2.697987	3.336595	0.638608
Longest Day, The (1962)	3.411765	4.031447	0.619682
Cable Guy, The (1996)	2.250000	2.863787	0.613787

Hình 13.2.3 Top 5 bộ phim nam giới thích hơn nữ giới.

Nhận xét:

Ngược lại, các bộ phim trong danh sách này được nam giới đánh giá cao hơn đáng kể. Đây thường là các bộ phim hành động, hài hước thô tục, hoặc phim chiến tranh, phản ánh sở thích của nam giới đối với các thể loại này.

4.4. Phân bố thể loại phim

Phân tích thể loại phim cho thấy các thể loại phổ biến nhất trong tập dữ liệu MovieLens 1M.

movie_id	title	genre
0	1	Toy Story (1995) Animation
0	1	Toy Story (1995) Children's
0	1	Toy Story (1995) Comedy
1	2	Jumanji (1995) Adventure
1	2	Jumanji (1995) Children's
1	2	Jumanji (1995) Fantasy
2	3	Grumpier Old Men (1995) Comedy
2	3	Grumpier Old Men (1995) Romance
3	4	Waiting to Exhale (1995) Comedy
3	4	Waiting to Exhale (1995) Drama

Hình 13.2.4. Phân bố top 10 thể loại phim được ưa thích

Nhận xét:

Thể loại Drama (Chính kịch) là phổ biến nhất, tiếp theo là Comedy (Hài) và Action (Hành động). Điều này cho thấy sự đa dạng trong sở thích xem phim của người dùng, với các thể loại truyền thống vẫn giữ vị trí quan trọng.

Các thể loại như Thriller, Sci-Fi, Romance cũng có số lượng đáng kể, phản ánh sự quan tâm của khán giả đối với nhiều loại hình điện ảnh khác nhau.

5. Kết luận

Đối với dữ liệu MovieLens 1M, chúng ta đã khám phá được sự khác biệt rõ rệt trong sở thích phim giữa nam và nữ giới, với nữ giới ưa chuộng các bộ phim lãng mạn/chính kịch và nam giới có xu hướng thích phim hành động/hài hước hơn. Thể loại Drama vẫn là thể loại phổ biến nhất trong tổng thể.

Những thông tin chi tiết này có thể hữu ích cho việc hiểu hành vi người dùng và phát triển các chiến lược nội dung hoặc tiếp thị mục tiêu.

III. US Baby Names 1880–2010

1. Giới thiệu

Bộ dữ liệu US Baby Names 1880–2010 được công bố bởi Cơ quan An sinh Xã hội Hoa Kỳ (Social Security Administration - SSA), ghi nhận tần suất xuất hiện của các tên trẻ sơ sinh tại Hoa Kỳ từ năm 1880 đến năm 2010. Mỗi bản ghi bao gồm tên, giới tính, năm sinh và số lượng trẻ được đặt tên đó trong năm tương ứng. Bộ dữ liệu này được sử dụng rộng rãi trong các nghiên cứu về xu hướng xã hội, ngôn ngữ học cũng như trong các ví dụ minh họa về xử lý và phân tích dữ liệu thời gian (time series) trong ngôn ngữ R và Python.

Bộ dữ liệu tham khảo [pydata-book/datasets/babynames at 3rd-edition](https://pydata-book/datasets/babynames-at-3rd-edition) · wesm/pydata-book.

2. Mục tiêu

Mục tiêu của việc phân tích bộ dữ liệu US Baby Names 1880–2010 là khám phá xu hướng đặt tên trẻ sơ sinh tại Hoa Kỳ trong hơn một thế kỷ, qua đó phản ánh sự thay đổi về văn hóa, xã hội và ngôn ngữ. Cụ thể:

- Xác định các tên phổ biến nhất theo từng năm và giới tính.

- Phân tích xu hướng tăng giảm của một số tên qua thời gian.

- So sánh sự khác biệt về tên gọi giữa nam và nữ.

- Ứng dụng các kỹ thuật xử lý và trực quan hóa dữ liệu thời gian (time series analysis) để thể hiện biến động theo năm.

3. Tổng quan dữ liệu

Bộ dữ liệu này bao gồm số liệu thống kê về tên trẻ em được đặt tại Hoa Kỳ từ năm 1880 đến năm 2010, được cung cấp bởi Cục An sinh Xã hội Hoa Kỳ. Mỗi bản ghi chứa tên, giới tính và số lượng trẻ sinh ra với tên đó trong một năm cụ thể. Bộ dữ liệu này rất hữu ích để phân tích xu hướng đặt tên, sự đa dạng của tên và các thay đổi văn hóa theo thời gian.

Mỗi bản ghi (record) trong tập dữ liệu tương ứng với một tên cụ thể được đặt trong một năm cho một giới tính nhất định, cùng với số lượng trẻ em nhận tên đó.

Các biến trong tập dữ liệu:

- name (tên trẻ): tên riêng của trẻ em đăng ký theo năm. Đây là biến định danh, dùng để phân tích mức độ phổ biến của từng tên.

- sex (giới tính): gồm hai giá trị chính “F” (Female: bé gái) và “M” (Male: bé trai). Biến này giúp phân biệt xu hướng đặt tên theo giới tính.

- births (Số lượng trẻ đặt tên đó): Là giá trị số thể hiện bao nhiêu trẻ trong năm đó mang tên tương ứng. Biến này rất quan trọng để tính mức độ phổ biến và xu hướng thay đổi của tên qua các năm.

- year (năm sinh): Thể hiện năm dữ liệu được ghi nhận, ví dụ: 1880, 1950, 2010... Giúp phân tích xu hướng thời gian: tên nào phổ biến trong từng giai đoạn lịch sử. → Biến được sinh ra khi gộp dữ liệu từ 1880-2010.

Ý nghĩa và ứng dụng:

Bộ dữ liệu này cung cấp cái nhìn toàn diện về sự thay đổi trong sở thích đặt tên của người Mỹ qua các thời kỳ.

Nó có thể được sử dụng để:

- Phân tích xu hướng đặt tên theo thời gian;

So sánh mức độ phổ biến giữa các giới tính;
Xác định các giai đoạn bùng nổ hoặc suy giảm của những cái tên cụ thể;
Khám phá ảnh hưởng văn hóa, lịch sử và truyền thông (ví dụ: tên phổ biến theo nhân vật nổi tiếng hoặc sự kiện xã hội).

4. Phân tích dữ liệu

4.1. Gộp tất cả các năm (1880-2010) vào một DataFrame

Việc gộp thành một bộ dữ liệu hoàn chỉnh giúp dễ dàng phân tích và đánh giá dữ liệu qua các năm.

```
pieces = []
for year in range(1880, 2011):
    path = f"datasets/babynames/yob{year}.txt"
    frame = pd.read_csv(path, names=["name", "sex", "births"])

    # Add a column for the year
    frame["year"] = year
    pieces.append(frame)

# Concatenate everything into a single DataFrame
names = pd.concat(pieces, ignore_index=True)
```

Python

Kết quả thực hiện:

	name	sex	births	year
0	Mary	F	7065	1880
1	Anna	F	2604	1880
2	Emma	F	2003	1880
3	Elizabeth	F	1939	1880
4	Minnie	F	1746	1880
...	...	...	...	...
1690779	Zymaire	M	5	2010
1690780	Zyonne	M	5	2010
1690781	Zyquarius	M	5	2010
1690782	Zyran	M	5	2010
1690783	Zzyzx	M	5	2010

1690784 rows × 4 columns

4.2. Phân tích theo tổng số sinh theo giới tính và năm sinh.

Mục tiêu chính là hiểu rõ sự thay đổi trong cơ cấu dân số theo giới tính qua các năm và đánh giá mức độ ổn định của tỷ lệ sinh giữa nam và nữ trong suốt hơn một thế kỷ tại Hoa Kỳ.



Hình 13.3.1 Biểu đồ đường mô tả sự phân bố tổng số ca sinh theo giới tính và năm

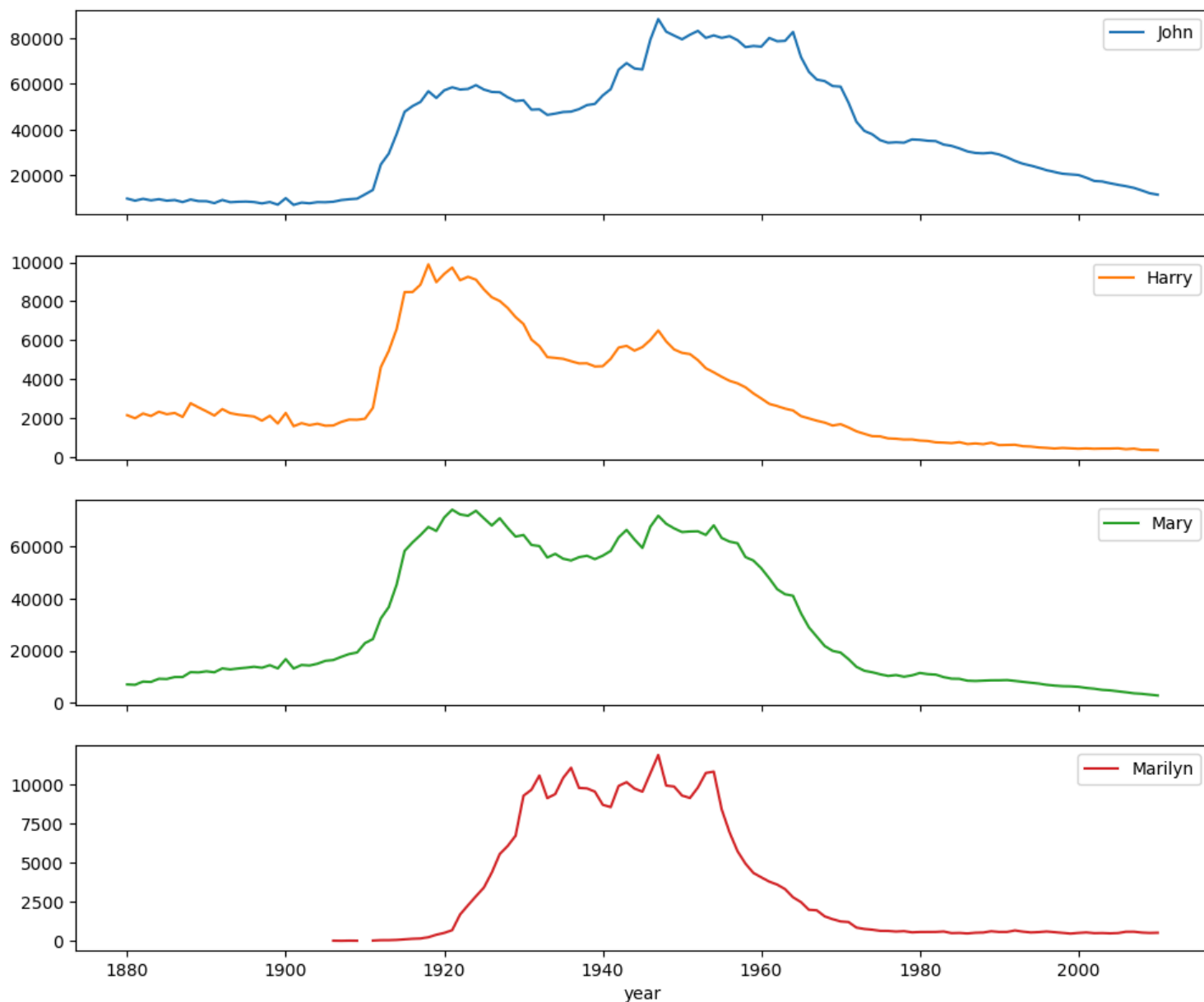
Nhận xét:

Biểu đồ này minh họa tổng số ca sinh theo giới tính (nam và nữ) qua các năm từ 1880 đến 2010. Có thể thấy rằng tổng số ca sinh tăng đều đặn theo thời gian, với một số biến động nhỏ. Số lượng bé trai sinh ra luôn cao hơn một chút so với bé gái trong suốt giai đoạn này, một xu hướng sinh học phổ biến trên toàn cầu.

4.3. Phân tích xu hướng một vài tên phổ biến.

Đánh giá một số tên phổ biến được nhất trong những năm (1880-2010).

Number of births per year



Biểu đồ đường mô tả xu hướng 4 cái tên phổ biến nhất trong những năm 1880-2010

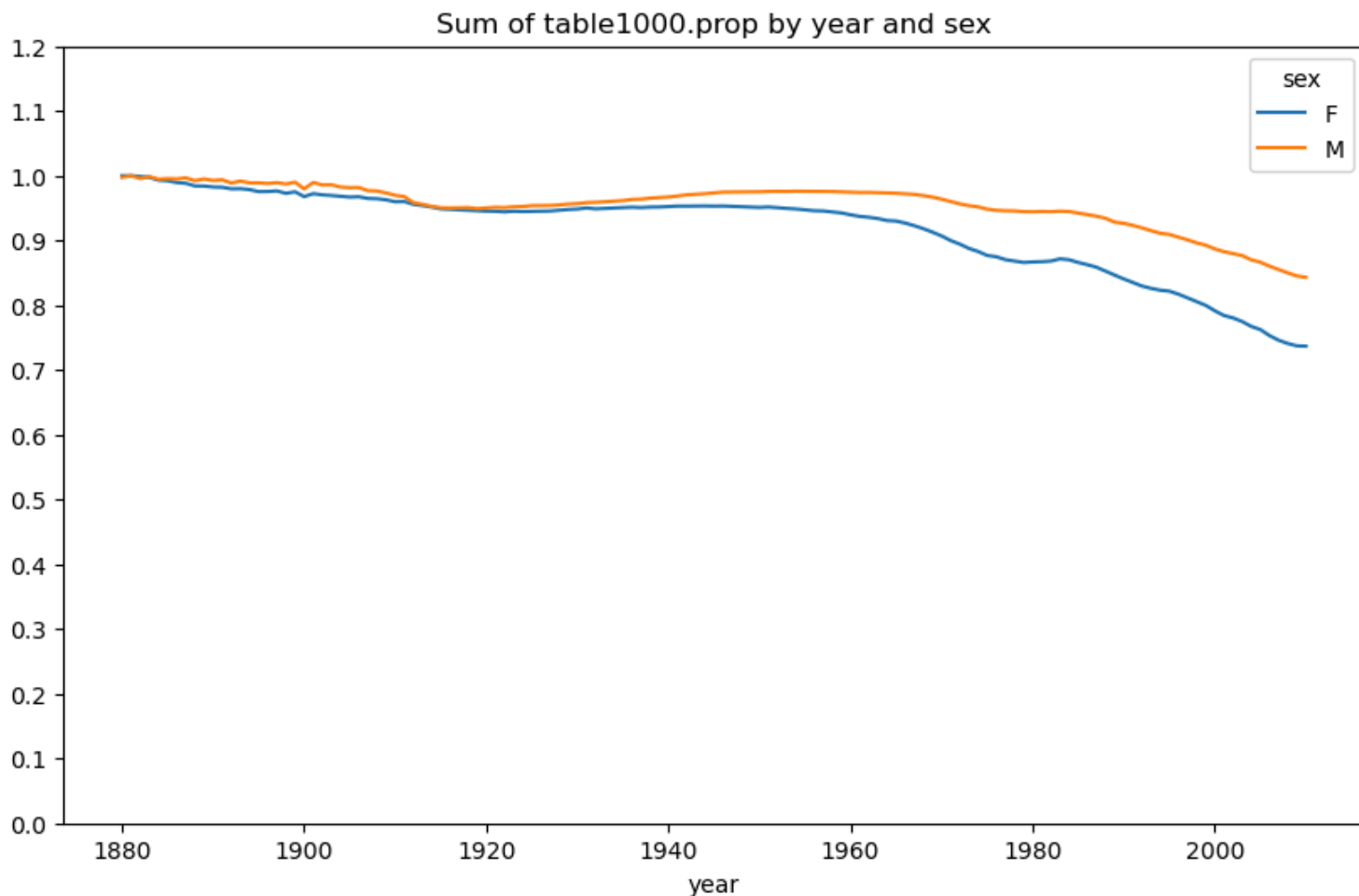
Nhận xét:

Biểu đồ này hiển thị xu hướng phổ biến của một số tên cụ thể ("John", "Harry", "Mary", "Marilyn") qua các năm. Tên "John" và "Mary" cho thấy sự phổ biến cao trong những năm đầu của thế kỷ 20 và dần giảm đi. Tên "Harry" có sự biến động nhưng duy trì ở mức tương đối ổn định. Tên "Marilyn" có sự tăng vọt vào giữa thế kỷ 20, có thể liên quan đến các yếu tố văn hóa hoặc người nổi tiếng, sau đó giảm dần. Biểu đồ này cho thấy sự thay đổi trong sở thích đặt tên theo thời gian.

4.4. Phân tích sự đa dạng của tên

Phân tích sự đa dạng tên giúp hiểu rõ hơn về sự thay đổi trong văn hóa đặt tên của người Mỹ qua hơn một thế kỷ, đồng thời phản ánh sự tiến hóa của quan niệm về cá tính, giới tính và bản sắc văn

hóa trong xã hội.



Hình 13.3.3 Biểu đồ đường đo lường mức độ đa dạng tên trên tổng số 1000 tên phổ biến nhất theo giới tính và năm sinh

Nhận xét:

Biểu đồ này thể hiện tổng tỷ lệ của 1000 tên phổ biến nhất theo giới tính và năm. Một giá trị gần 1 cho thấy rằng 100 tên hàng đầu chiếm phần lớn tổng số ca sinh, trong khi giá trị thấp hơn cho thấy sự đa dạng tên lớn hơn. Biểu đồ cho thấy rằng sự đa dạng của tên đã tăng lên đáng kể theo thời gian, đặc biệt là sau giữa thế kỷ 20. Điều này có nghĩa là ngày càng có nhiều tên khác nhau được sử dụng, và 1000 tên hàng đầu chiếm một tỷ lệ nhỏ hơn trong tổng số ca sinh.

4.5. Tỷ lệ bé trai có tên kết thúc bằng d/n/y theo thời gian

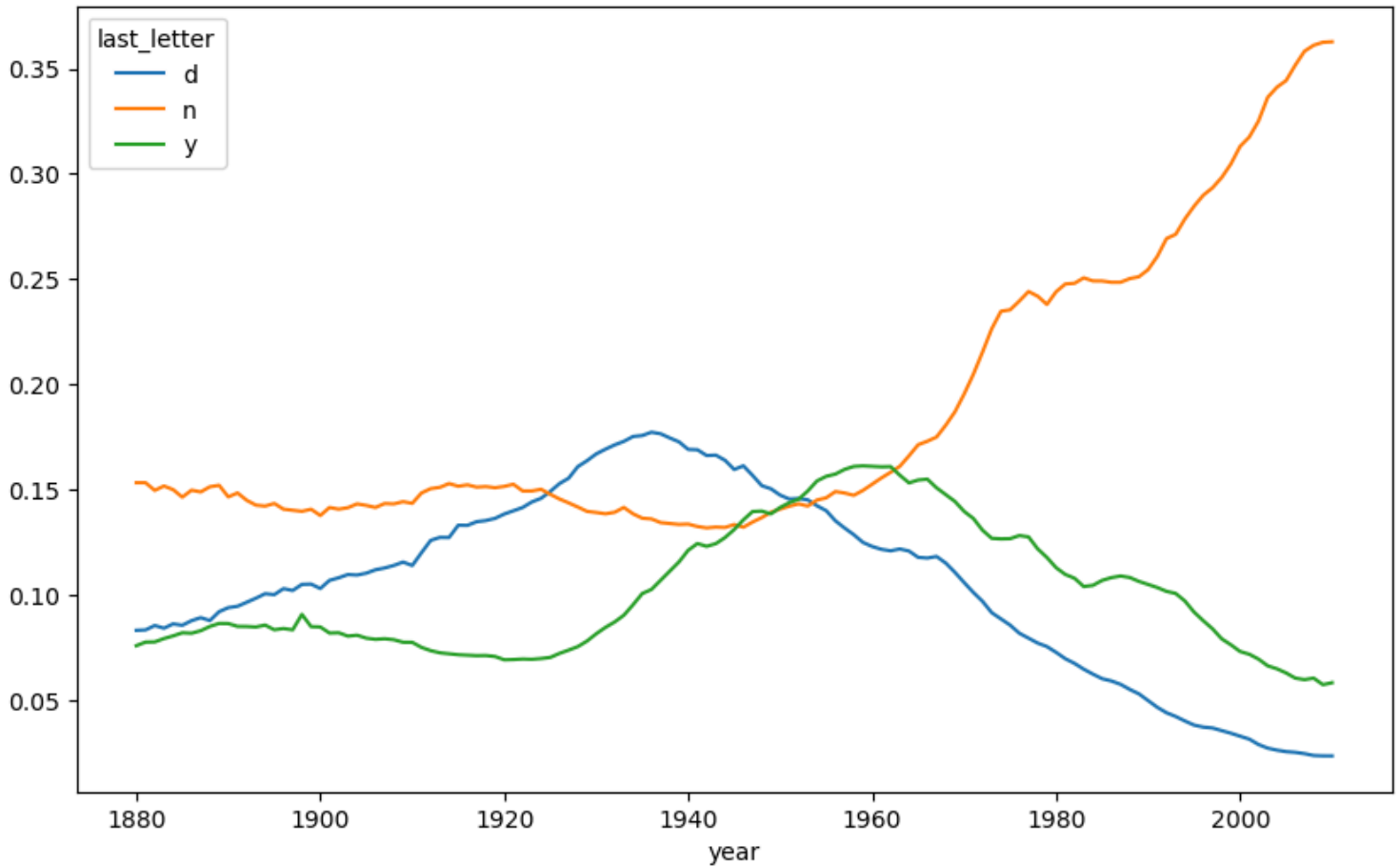
Các chữ cái cuối trong tên thường mang đặc trưng âm tiết và cảm nhận giới tính trong ngôn ngữ. Do đó, việc theo dõi sự thay đổi của chúng giúp:

Phát hiện xu hướng phát âm ưa chuộng trong các thời kỳ, ví dụ như tên kết thúc bằng “n” (Jason, Ryan, Benjamin) trở nên phổ biến hơn trong thế kỷ 20.

Phân tích sự biến đổi trong phong cách đặt tên cho nam giới, từ những tên cổ điển (kết thúc bằng “d”: David, Richard) sang những tên hiện đại hoặc nhẹ nhàng hơn (kết thúc bằng “y”: Anthony, Jeremy).

Phản ánh ảnh hưởng của văn hóa đại chúng, chẳng hạn như nhân vật trong phim ảnh, truyền hình hoặc âm nhạc có thể làm tăng độ phổ biến của những kiểu tên nhất định.

So sánh xu hướng âm cuối giữa các giai đoạn lịch sử, giúp hiểu rõ hơn sự thay đổi trong thị hiếu ngôn ngữ và xã hội.



Hình 13.3.4 Biểu đồ đường đánh giá số lượng tên bé trai kết thúc bằng d/n/y qua từng năm

Nhận xét:

Biểu đồ này phân tích xu hướng của các tên bé trai kết thúc bằng các chữ cái 'd', 'n', 'y' qua các năm. Có thể thấy rằng tỷ lệ tên kết thúc bằng 'n' đã tăng lên đáng kể theo thời gian, trở thành một trong những hậu tố phổ biến nhất. Ngược lại, tỷ lệ tên kết thúc bằng 'd' và 'y' có xu hướng giảm hoặc duy trì ở mức thấp hơn. Điều này phản ánh sự thay đổi trong các mẫu âm thanh và cấu trúc tên được ưa chuộng cho bé trai.

5. Kết luận

Dữ liệu tên trẻ em tiết lộ sự tăng trưởng tổng thể của dân số, sự thay đổi trong phổ biến của các tên cụ thể và xu hướng tăng đa dạng tên theo thời gian. Những phân tích này không chỉ cung cấp cái nhìn về các mẫu hình lịch sử mà còn có thể được sử dụng để dự đoán các xu hướng trong tương lai và đưa ra các quyết định dựa trên dữ liệu.

IV. Cơ sở dữ liệu thực phẩm USDA

1. Giới thiệu

Trong notebook này, chúng ta sẽ khám phá Cơ sở dữ liệu Thực phẩm của Bộ Nông nghiệp Hoa Kỳ (USDA), chứa thông tin dinh dưỡng chi tiết cho hàng ngàn loại thực phẩm.

Sử dụng các thư viện Python như pandas, json, matplotlib, và seaborn, chúng ta sẽ:

- Tải và xử lý dữ liệu từ định dạng JSON.
- Gộp và làm sạch dữ liệu.
- Phân tích các nhóm thực phẩm theo giá trị dinh dưỡng.
- Trực quan hóa mối tương quan giữa các chất dinh dưỡng.

2. Mục tiêu

Mục tiêu của dự án này là khám phá, phân tích và trực quan hóa thành phần dinh dưỡng của các nhóm thực phẩm khác nhau từ bộ dữ liệu của USDA. Dữ liệu dinh dưỡng đóng vai trò nền tảng trong nhiều lĩnh vực, từ nghiên cứu sức khỏe cộng đồng, y học, đến việc xây dựng chế độ ăn uống khoa học cho cá nhân. Bằng cách phân tích bộ dữ liệu này, chúng ta có thể xác định các mô hình dinh dưỡng, so sánh các nhóm thực phẩm và tìm ra những nguồn dưỡng chất quan trọng, từ đó cung cấp một cơ sở bằng chứng vững chắc cho các quyết định liên quan đến sức khỏe và dinh dưỡng.

3. Tổng quan dữ liệu.

3.1. Chuẩn bị dữ liệu.

Quy trình chuẩn bị dữ liệu là bước khởi đầu quan trọng để đảm bảo tính chính xác và hiệu quả cho các phân tích sau này.

Đọc dữ liệu JSON

```
db=json.load(open("database_usa_food.json"))  
print("Tổng số loại thực phẩm : ",len(db))
```

Tổng số loại thực phẩm : 6636

Dữ liệu ban đầu được cung cấp dưới định dạng JSON, bao gồm thông tin chi tiết về 6,636 loại thực phẩm. Mỗi bản ghi thực phẩm chứa các thông tin mô tả cơ bản và một danh sách chi tiết các thành phần dinh dưỡng.

3.2 Tiền xử lý dữ liệu.

3.2.1 Trích xuất thông tin.

```
info_keys = ['description', 'group', 'id', 'manufacturer']
info = pd.DataFrame(db, columns=info_keys)

print("\n--- Thông tin cơ bản ---")
print(info.head())
```

--- Thông tin cơ bản ---

	description	group	id \
0	Cheese, caraway	Dairy and Egg Products	1008
1	Cheese, cheddar	Dairy and Egg Products	1009
2	Cheese, edam	Dairy and Egg Products	1018
3	Cheese, feta	Dairy and Egg Products	1019
4	Cheese, mozzarella, part skim milk	Dairy and Egg Products	1028

manufacturer

0
1
2
3
4

```
nutrients_list = []
for food in db:
    fnuts = pd.DataFrame(food['nutrients'])
    fnuts['id'] = food['id']
    nutrients_list.append(fnuts)

nutrients = pd.concat(nutrients_list, ignore_index=True)

print(f"Tổng số bản ghi dinh dưỡng: {len(nutrients):,}")
nutrients.head()
```

Tổng số bản ghi dinh dưỡng: 389,355

Dữ liệu được tách thành hai phần riêng biệt để dễ dàng xử lý. Thông tin cơ bản của thực phẩm (tên, nhóm, ID) được đưa vào một DataFrame, trong khi thông tin dinh dưỡng chi tiết (giá trị, đơn vị, tên chất) được tổng hợp vào một DataFrame khác. Quá trình này đã tạo ra tổng cộng 389,355 bản ghi dinh dưỡng từ tất cả các loại thực phẩm.

3.2.2 Gộp dữ liệu.

```
ndata = pd.merge(nutrients, info, on='id', how='outer')
```

Xem cấu trúc dữ liệu

```
ndata.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 389355 entries, 0 to 389354
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   value           389355 non-null float64
1   units           389355 non-null object
2   description_x    389355 non-null object
3   group_x         389355 non-null object
4   id              389355 non-null int64
5   description_y    389355 non-null object
6   group_y         389355 non-null object
7   manufacturer     305162 non-null object
dtypes: float64(1), int64(1), object(6)
memory usage: 23.8+ MB
```

Hai DataFrame trên đã được hợp nhất (merge) dựa trên ID chung của thực phẩm. Bước này tạo ra một bộ dữ liệu tổng hợp, trong đó mỗi bản ghi về chất dinh dưỡng được liên kết trực tiếp với thông tin chi tiết của thực phẩm tương ứng, tạo điều kiện thuận lợi cho việc phân tích toàn diện.

3.2.3 Làm sạch dữ liệu

```
ndata = ndata.drop_duplicates(subset=['description_x', 'group_x', 'id', 'value'])

ndata = ndata.rename(columns={
    'description_x': 'chat_dinh_duong',
    'group_x': 'nhom_chat',
    'description_y': 'thuc_pham',
    'group_y': 'nhom_thuc_pham'
})

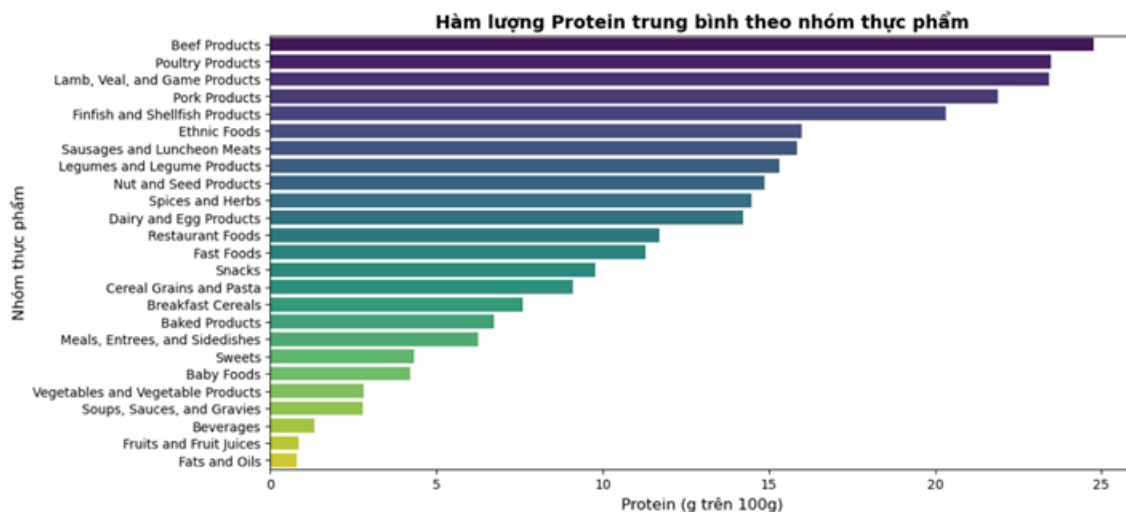
ndata.head()
```

	value	units	chat_dinh_duong	nhom_chat	id	thuc_pham	nhom_thuc_pham	manufacturer
0	25.18	g	Protein	Composition	1008	Cheese, caraway	Dairy and Egg Products	
1	29.20	g	Total lipid (fat)	Composition	1008	Cheese, caraway	Dairy and Egg Products	
2	3.06	g	Carbohydrate, by difference	Composition	1008	Cheese, caraway	Dairy and Egg Products	
3	3.28	g	Ash	Other	1008	Cheese, caraway	Dairy and Egg Products	
4	376.00	kcal	Energy	Energy	1008	Cheese, caraway	Dairy and Egg Products	

Các bước làm sạch dữ liệu đã được tiến hành để nâng cao chất lượng dữ liệu. Các bản ghi trùng lặp đã được xác định và loại bỏ. Đồng thời, tên các cột đã được chuẩn hóa và đổi sang tiếng Việt để tăng tính tường minh và dễ hiểu (ví dụ: description_x được đổi thành chat_dinh_duong, group_y thành nhom_thuc_pham).

3.3 Phân tích và Trực quan hóa dữ liệu.

3.3.1 Hàm lượng Dinh dưỡng Trung bình theo Nhóm Thực phẩm



- Biểu đồ thể hiện trực quan nhóm thực phẩm nào giàu Protein nhất.
- Dễ dàng nhận thấy các nhóm như thịt, sữa, đậu có giá trị Protein cao vượt trội.

Một trong những phân tích cơ bản là tính toán hàm lượng dinh dưỡng trung bình cho mỗi nhóm thực phẩm. Khi xem xét hàm lượng Protein, kết quả cho thấy một sự khác biệt rõ rệt giữa các

nhóm. Các nhóm như Legumes and Legume Products (Các loại Đậu và Sản phẩm từ Đậu) và Dairy and Egg Products (Sản phẩm từ Sữa và Trứng) có hàm lượng Protein trung bình cao vượt trội so với các nhóm khác. Phát hiện này khẳng định vai trò quan trọng của các nhóm thực phẩm này trong việc cung cấp Protein cho cơ thể.

3.3.2 Xác định các Thực phẩm Giàu Dinh dưỡng Hàng đầu

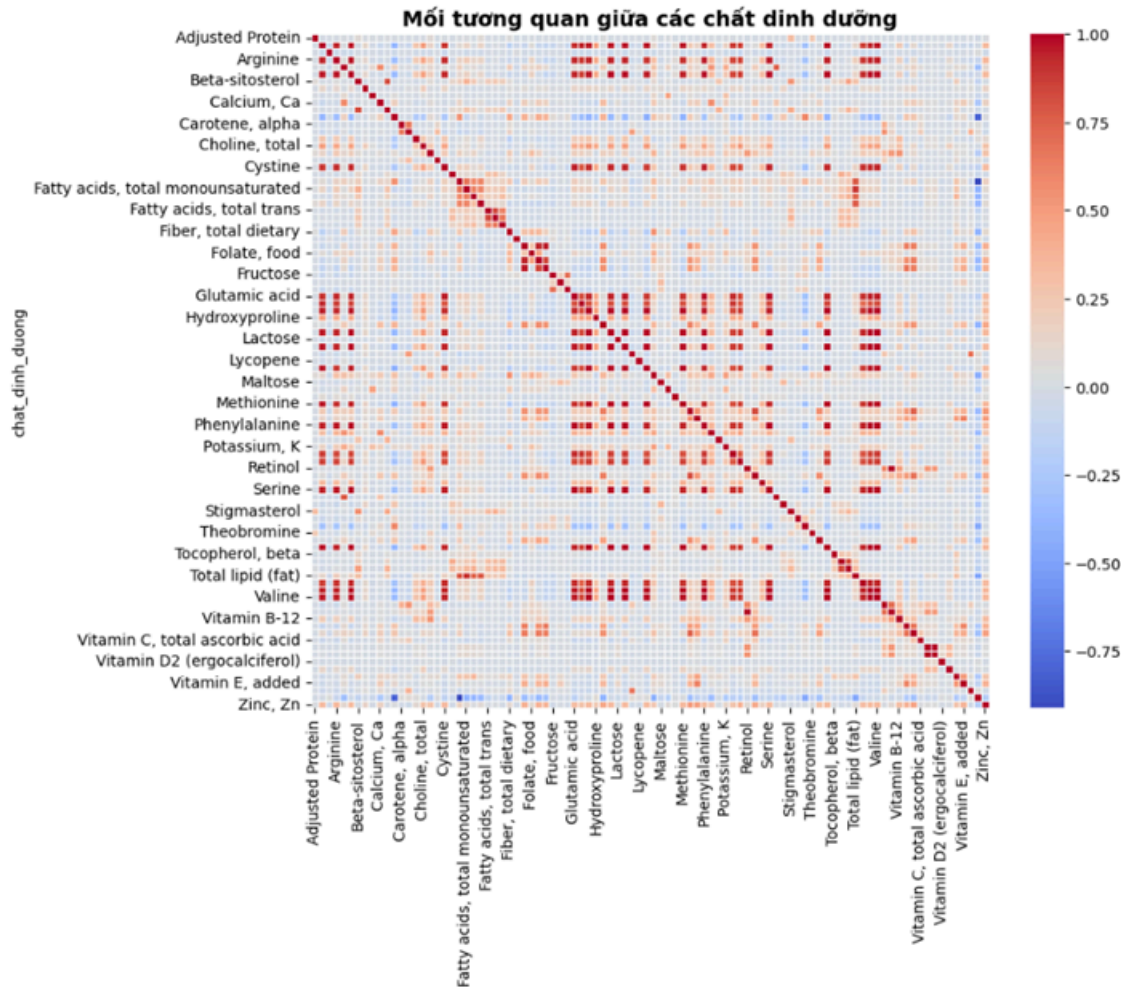
Top 10 thực phẩm chứa nhiều Protein nhất:

	thuc_pham	value	units	\
0	Soy protein isolate, potassium type, crude pro...	88.32	g	
1	Soy protein isolate, PROTEIN TECHNOLOGIES INTE...	87.75	g	
2	Soy protein isolate, PROTEIN TECHNOLOGIES INTE...	86.00	g	
3	Gelatins, dry powder, unsweetened	85.60	g	
4	Seal, bearded (Oogruk), meat, dried (Alaska Na...	82.60	g	
5	Soy protein isolate	80.69	g	
6	Soy protein isolate, potassium type	80.69	g	
7	Steelhead trout, dried, flesh (Shoshone Bannock)	77.27	g	
8	Vital wheat gluten	75.16	g	
9	Whale, beluga, meat, dried (Alaska Native)	69.86	g	

	nhom_thuc_pham
0	Legumes and Legume Products
1	Legumes and Legume Products
2	Legumes and Legume Products
3	Sweets
4	Ethnic Foods
5	Legumes and Legume Products
6	Legumes and Legume Products
7	Ethnic Foods
8	Cereal Grains and Pasta
9	Ethnic Foods

Kết quả này có ý nghĩa thực tiễn cao, giúp xác định nhanh chóng các nguồn protein tinh khiết và đậm đặc. Thông tin này đặc biệt hữu ích trong lĩnh vực tư vấn dinh dưỡng, thể hình, hoặc trong việc phát triển các sản phẩm thực phẩm chức năng.

3.3.3 Mối tương quan giữa các Chất dinh dưỡng



Phân tích tương quan giúp khám phá mối quan hệ nội tại giữa các chất dinh dưỡng trong thực phẩm. Biểu đồ nhiệt tương quan cho thấy một số cặp chất dinh dưỡng thường xuất hiện cùng nhau. Điển hình là tương quan thuận mạnh mẽ giữa chất béo (Total lipid) và năng lượng (Energy). Về mặt sinh hóa, điều này hoàn toàn hợp lý vì chất béo là đại dưỡng chất có mật độ năng lượng cao nhất (cung cấp khoảng 9 kcal mỗi gram). Việc hiểu rõ các mối liên kết này giúp xây dựng một cái nhìn toàn diện hơn về thành phần dinh dưỡng và cách chúng ảnh hưởng lẫn nhau trong một chế độ ăn uống.

4. Kết luận

Dự án đã thực hiện thành công quy trình phân tích dữ liệu dinh dưỡng từ A đến Z, bắt đầu từ việc tải và tiền xử lý dữ liệu JSON phức tạp, sau đó gộp, làm sạch, và cuối cùng là phân tích và trực quan hóa để rút ra thông tin hữu ích. Những hiểu biết chính thu được bao gồm việc xác định các nhóm thực phẩm giàu protein, liệt kê các thực phẩm cụ thể chứa hàm lượng protein cao nhất, và khám phá mối quan hệ tương quan giữa các chất dinh dưỡng. Phân tích này đã cung cấp một cái

nhìn sâu sắc về thành phần dinh dưỡng của thực phẩm, tạo nền tảng vững chắc cho các ứng dụng thực tế trong lĩnh vực sức khỏe và xây dựng chế độ ăn uống khoa học. Tiếp theo, báo cáo sẽ chuyển sang dự án thứ hai với một bộ dữ liệu và bối cảnh hoàn toàn khác: phân tích dữ liệu bầu cử.

V. Phân tích Đóng góp cho Cuộc Bầu cử Tổng thống Hoa Kỳ năm 2012

1. Giới thiệu

Dữ liệu từ Ủy ban Bầu cử Liên bang Hoa Kỳ (Federal Election Commission - FEC) năm 2012 chứa thông tin về các khoản đóng góp cho các ứng viên tổng thống.

Trong phần này, chúng ta sẽ khám phá dữ liệu này, làm sạch, xử lý, và phân tích các khoản đóng góp theo ứng viên, đảng phái và tiểu bang.

2. Mục tiêu.

Trong dự án thứ hai, chúng ta sẽ chuyển sang lĩnh vực khoa học xã hội và chính trị. Mục tiêu của dự án này là phân tích dữ liệu các khoản đóng góp cho chiến dịch tranh cử từ Ủy ban Bầu cử Liên bang (FEC). Bằng cách khám phá dữ liệu này, chúng ta có thể tìm hiểu các xu hướng và mô hình gây quỹ của hai ứng cử viên tổng thống chính trong cuộc bầu cử năm 2012 là Barack Obama (Đảng Dân chủ) và Mitt Romney (Đảng Cộng hòa). Phân tích sẽ tập trung vào các khía cạnh như tổng số tiền đóng góp, sự khác biệt theo nghề nghiệp và phân bố địa lý, từ đó cung cấp một cái nhìn dựa trên dữ liệu về sự ủng hộ tài chính cho mỗi ứng viên.

3. Tổng quan dữ liệu.

3.1 Chuẩn bị dữ liệu

US 13.5 — 2012 Federal Election Commission Database

- Tập dữ liệu: P00000001-ALL.csv chứa hơn 1 triệu bản ghi đóng góp cá nhân.
- Mỗi bản ghi gồm các cột: cand_nm (tên ứng viên), contbr_nm (người đóng góp), contbr_st (bang), contbr_employer (nơi làm việc), contbr_occupation (nghề nghiệp), contb_receipt_amt (số tiền đóng góp)...

```
[5]: fec = pd.read_csv('P00000001-ALL.csv')  
  
print(f'Tổng số bản ghi: {len(fec):,}')  
print(f'Tổng số cột: {fec.shape[1]}')
```

Tổng số bản ghi: 536,041
Tổng số cột: 16

Bộ dữ liệu được sử dụng trong dự án này là một tập CSV chứa thông tin chi tiết về các khoản đóng góp cá nhân, với tổng số 536,041 bản ghi ban đầu.

3.2 Tiền xử lý dữ liệu

3.2.1. Xử lý trùng lặp

Kiểm tra trùng lặp

```
fec.duplicated().sum()
```

16065

🔍 Nhận Xét:

- Số lượng bản ghi trùng lặp rất lớn cần được loại bỏ

Xử lý trùng lặp

```
fec=fec.drop_duplicates()
```

```
fec.duplicated().sum()
```

0

Quá trình kiểm tra ban đầu đã phát hiện 16,065 bản ghi trùng lặp. Các bản ghi này đã được loại bỏ để tránh làm sai lệch kết quả phân tích.

3.2.2. Lọc dữ liệu

Dữ liệu chứa cả các khoản đóng góp có giá trị âm, đại diện cho các khoản hoàn tiền. Những bản ghi này đã được lọc ra để chỉ tập trung vào các khoản đóng góp thực tế, phản ánh đúng dòng tiền ủng hộ cho các ứng viên.

3.2.3 Thêm thông tin Đảng phái.

Tạo từ điển ánh xạ giữa ứng viên và đảng phái

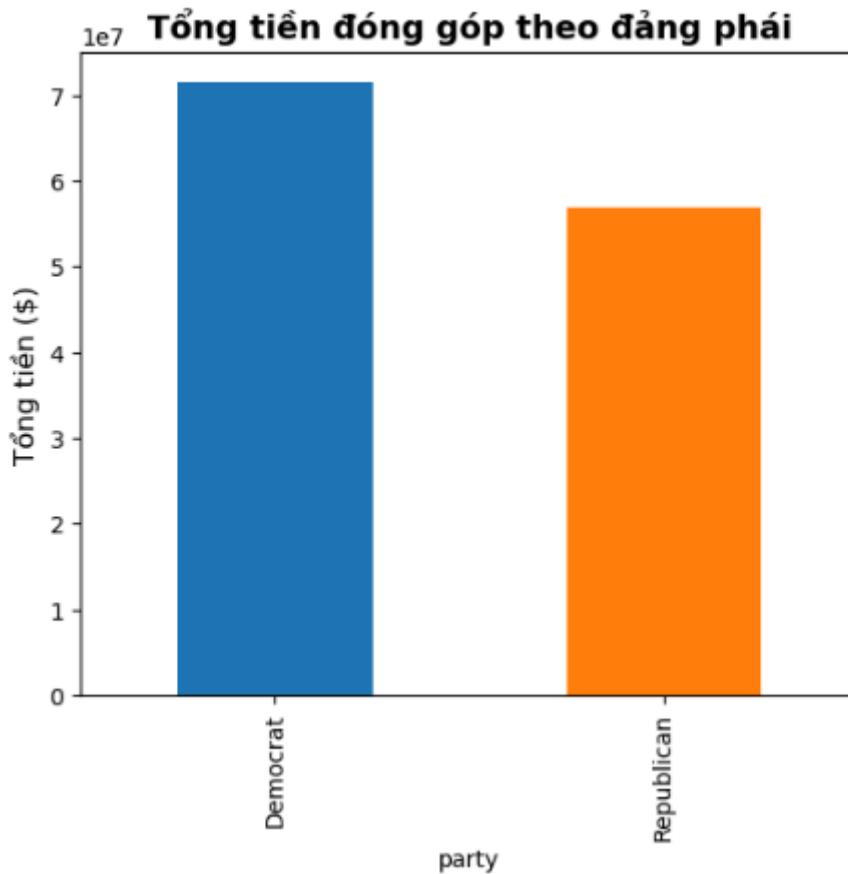
```
parties = {  
    'Obama, Barack': 'Democrat',  
    'Romney, Mitt': 'Republican'  
}  
fec['party'] = fec['cand_nm'].map(parties)
```

Để thuận tiện cho việc so sánh, một cột mới có tên party đã được tạo ra. Cột này được điền thông tin bằng cách ánh xạ tên của hai ứng viên chính ('Obama, Barack' và 'Romney, Mitt') với đảng phái tương ứng của họ ('Democrat' và 'Republican').

Sau khi hoàn thành các bước làm sạch và chuẩn hóa, bộ dữ liệu đã được cấu trúc lại một cách tối ưu, tập trung vào việc so sánh trực tiếp giữa hai đảng phái lớn, sẵn sàng cho giai đoạn phân tích và trực quan hóa.

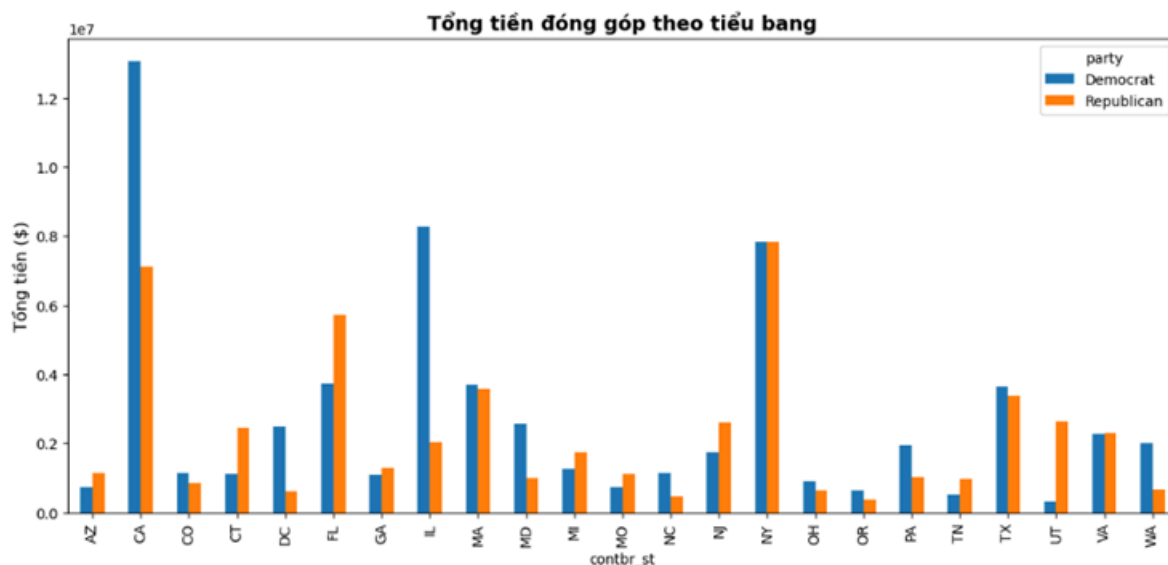
3.4 Phân tích và trực quan hóa dữ liệu.

3.4.1 Phân tích Tổng đóng góp theo Đảng phái.



Kết quả tổng hợp cho thấy một sự khác biệt đáng kể về tổng số tiền gây quỹ giữa hai đảng. Cụ thể, Đảng Dân chủ (đại diện là Barack Obama) đã nhận được tổng cộng 56,908,991. Phân tích sâu hơn cho thấy Đảng Dân chủ có xu hướng nhận được nhiều khoản đóng góp với giá trị nhỏ từ một số lượng lớn các cá nhân. Điều này cho thấy một chiến dịch gây quỹ "quần chúng" (grassroots) thành công, phản ánh sự ủng hộ rộng rãi từ các cá nhân, trong khi các khoản đóng góp lớn hơn với tần suất thấp hơn có thể gợi ý sự phụ thuộc vào các nhóm nhà tài trợ khác.

3.4.2 Phân tích Đóng góp theo Tiểu bang



Phân tích theo địa lý cho thấy các bang đông dân và có nền kinh tế lớn như California, New York và Illinois là những nơi có tổng số tiền đóng góp cao nhất. Đặc biệt, trường hợp của California rất đáng chú ý: bang này đã đóng góp cho chiến dịch của Đảng Dân chủ hơn 1,2 triệu đô, gần gấp đôi so với khoản đóng góp cho Đảng Cộng hòa. Điều này không chỉ cho thấy California là một "thành trì" tài chính của Đảng Dân chủ mà còn minh họa một xu hướng chính trị địa lý rõ ràng, nơi sự ủng hộ của cử tri được thể hiện mạnh mẽ qua các đóng góp tài chính.

4. Kết Luận

Phân tích dữ liệu đóng góp cho cuộc bầu cử năm 2012 đã phát hiện ra nhiều mô hình quan trọng. Các kết quả chính cho thấy Đảng Dân chủ dẫn trước về tổng số tiền gây quỹ, với sự ủng hộ mạnh mẽ từ các nhóm nghề nghiệp cụ thể như người đã nghỉ hưu và có lợi thế áp đảo ở các bang chiến lược như California. Phân tích này cho thấy xu hướng ủng hộ của cử tri dành cho Đảng Dân chủ cao hơn về tổng thể, không chỉ về số lượng đóng góp mà còn về phân bố địa lý. Những mô hình tài chính này cung cấp một chỉ báo mạnh mẽ, phản ánh động lực ủng hộ của cử tri và là một yếu tố then chốt giúp lý giải kết quả cuối cùng của cuộc bầu cử. Những phân tích này sẽ được tổng hợp trong phần kết luận chung của báo cáo.

Kết quả tổng hợp cho thấy một sự khác biệt đáng kể về tổng số tiền gây quỹ giữa hai đảng. Cụ thể, Đảng Dân chủ (đại diện là Barack Obama) đã nhận được tổng cộng 56,908,991. Phân tích sâu hơn cho thấy Đảng Dân chủ có xu hướng nhận được nhiều khoản đóng góp với giá trị nhỏ từ một số lượng lớn các cá nhân. Điều này cho thấy một chiến dịch gây quỹ "quần chúng" (grassroots) thành công, phản ánh sự ủng hộ rộng rãi từ các cá nhân, trong khi các khoản đóng góp lớn hơn với tần suất thấp hơn có thể gợi ý sự phụ thuộc vào các nhóm nhà tài trợ khác.

Phụ lục A : Numpy Nâng cao

Giới thiệu

Phụ lục này sẽ đi sâu hơn vào thư viện NumPy cho việc tính toán mảng. Nội dung sẽ bao gồm các chi tiết nội tại về kiểu ndarray và các thao tác, thuật toán mảng nâng cao hơn. Phụ lục này chứa các chủ đề đa dạng và không nhất thiết phải đọc theo trình tự. Xuyên suốt các chương, chúng tôi sẽ tạo dữ liệu ngẫu nhiên cho nhiều ví dụ bằng cách sử dụng trình tạo số ngẫu nhiên mặc định trong module `numpy.random`.

```
import numpy as np
import pandas as pd
rng = np.random.default_rng(seed=12345)
```

A.1. Cấu trúc nội tại của đối tượng ndarray (ndarray Object Internals)

ndarray của NumPy cung cấp một cách dễ diễn giải một khối dữ liệu có kiểu đồng nhất (liên kết hoặc có bước nhảy) như một đối tượng mảng đa chiều. Kiểu dữ liệu, hay dtype, quyết định cách dữ liệu được diễn giải là số thực, số nguyên, Boolean, hoặc bất kỳ kiểu nào khác mà chúng ta đã xem

xét.

Một phần làm cho ndarray trở nên linh hoạt là mỗi đối tượng mảng đều là một **khung nhìn có bước nhảy (strided view)** trên một khối dữ liệu. Bạn có thể tự hỏi, ví dụ, làm thế nào mà khung nhìn `arr[:,2, :-1]` không sao chép bất kỳ dữ liệu nào. Lý do là ndarray không chỉ là một khối bộ nhớ và một kiểu dữ liệu; nó còn có thông tin về **bước nhảy (striding)** cho phép mảng di chuyển qua bộ nhớ với các kích thước bước khác nhau. Cụ thể hơn, ndarray bên trong bao gồm:

Một con trỏ đến dữ liệu (pointer to data) — tức là, một khối dữ liệu trong RAM hoặc trong một tệp được ánh xạ bộ nhớ (memory-mapped file).

Kiểu dữ liệu (data type) hay dtype mô tả các ô giá trị có kích thước cố định trong mảng.

Một tuple cho biết hình dạng (shape) của mảng.

Một tuple chứa các bước nhảy (strides) — các số nguyên cho biết số byte cần "bước" để tiến tới phần tử tiếp theo dọc theo một chiều.

See [Figure A-1](#) for a simple mock-up of the ndarray innards.

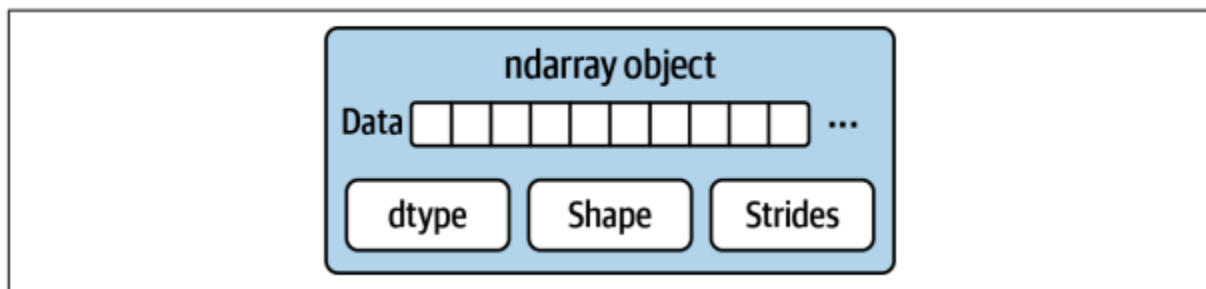


Figure A-1. The NumPy ndarray object

Ví dụ, một mảng 10×5 sẽ có hình dạng `(10, 5)`:

```
# For example, a 10 x 5 array would have the shape (10, 5):  
np.ones((10, 5)).shape
```

```
(10, 5)
```

Một mảng $3 \times 4 \times 5$ diễn hình (thứ tự C) của các giá trị float64 (8-byte) có các bước nhảy là `(160, 40, 8)`. Việc biết về các bước nhảy có thể hữu ích vì, nói chung, bước nhảy trên một trục càng lớn thì việc thực hiện tính toán dọc theo trục đó càng tốn kém.


```
# A typical (C order) 3 x 4 x 5 array
np.ones((3, 4, 5), dtype=np.float64).strides
```

```
(160, 40, 8)
```

Mặc dù hiếm khi người dùng NumPy thông thường quan tâm đến các bước nhảy của mảng, chúng lại cần thiết để xây dựng các khung nhìn mảng "không sao chép" (zero-copy). Các bước nhảy thậm chí có thể là số âm, cho phép một mảng di chuyển "lùi" qua bộ nhớ (ví dụ như trong một lát cắt `obj[::-1]` hoặc `obj[:, ::-1]`).

Phân cấp kiểu dữ liệu NumPy

Đôi khi, bạn có thể có mã cần kiểm tra xem một mảng có chứa số nguyên, số dấu phẩy động, chuỗi hoặc đối tượng Python hay không. Bởi vì có nhiều loại số dấu phẩy động (float16 đến float128), việc kiểm tra xem loại dữ liệu có nằm trong danh sách các loại sẽ rất dài dòng hay không. May mắn thay, các kiểu dữ liệu có siêu lớp, chẳng hạn như `np.integer` và `np.floating`, có thể được sử dụng với hàm `np.issubdtype`:

```
ints = np.ones(10, dtype=np.uint16)
floats = np.ones(10, dtype=np.float32)
np.issubdtype(ints.dtype, np.integer)
```

```
True
```

```
np.issubdtype(floats.dtype, np.floating)
```

```
True
```

Bạn có thể xem tất cả các lớp cha của một kiểu dữ liệu cụ thể bằng cách gọi phương thức `mro` của kiểu đó:

```
np.float64.mro()
```

```
[numpy.float64,
 numpy.floating,
 numpy.inexact,
 numpy.number,
 numpy.generic,
 float,
 object]
```

Vì vậy, chúng tôi cũng có:

```
np.issubdtype(ints.dtype, np.number)
```

True

Hầu hết người dùng NumPy sẽ không bao giờ biết về điều này, nhưng đôi khi nó rất hữu ích. Xem Hình A-2 để biết biểu đồ về phân cấp kiểu dữ liệu và mối quan hệ lớp con-lớp cha.

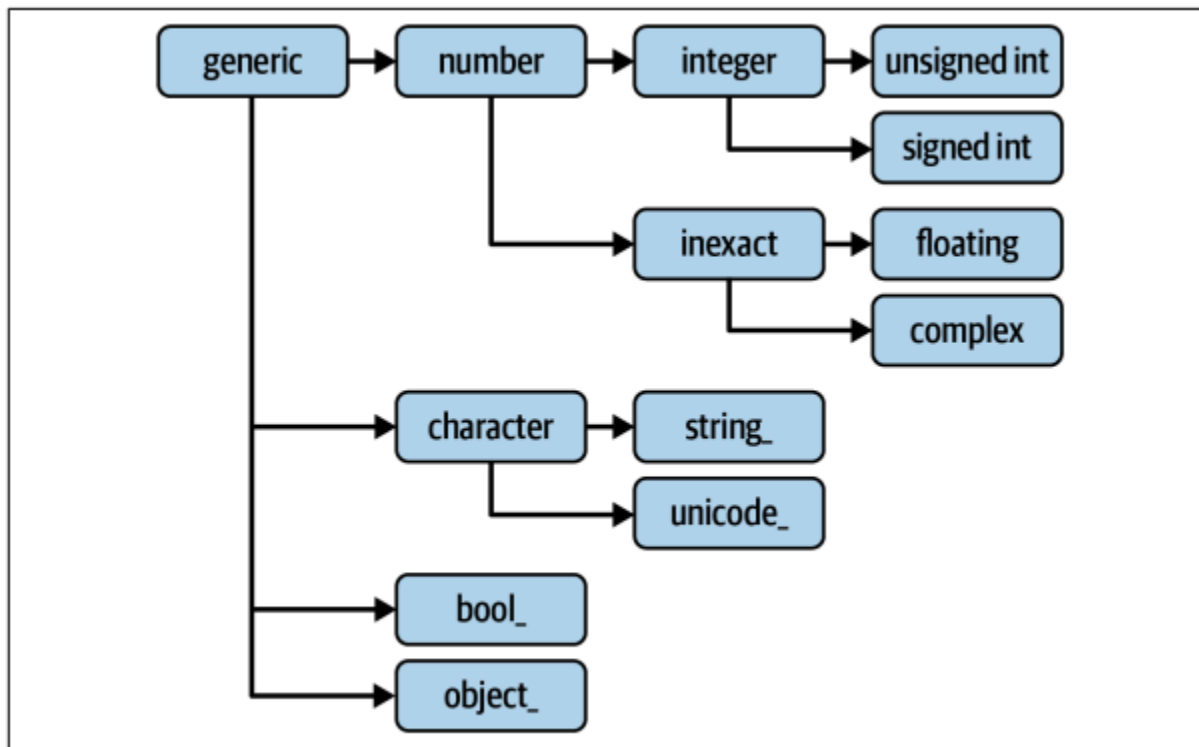


Figure A-2. The NumPy data type class hierarchy

A.2. Thao tác Mảng Nâng cao (Advanced Array Manipulation)

Có nhiều cách để làm việc với mảng ngoài việc lập chỉ mục, cắt lát và tập hợp con Boolean. Mặc dù phần lớn công việc nặng nhọc cho các ứng dụng phân tích dữ liệu được xử lý bởi các hàm cấp cao hơn trong cấu trúc, nhưng đôi khi bạn có thể cần phải viết một thuật toán dữ liệu không có trong một trong các thư viện hiện có.

Thay đổi định dạng Mảng (Reshaping Arrays)

Trong nhiều trường hợp, bạn có thể chuyển đổi một mảng từ hình dạng này sang hình dạng khác mà không cần sao chép dữ liệu. Để làm điều này, hãy truyền một tuple chỉ định hình dạng mới vào phương thức reshape của mảng.

```
arr = np.arange(8)
arr
```

```
array([0, 1, 2, 3, 4, 5, 6, 7])
```

```
arr.reshape((4, 2))
```

```
array([[0, 1],
       [2, 3],
       [4, 5],
       [6, 7]])
```

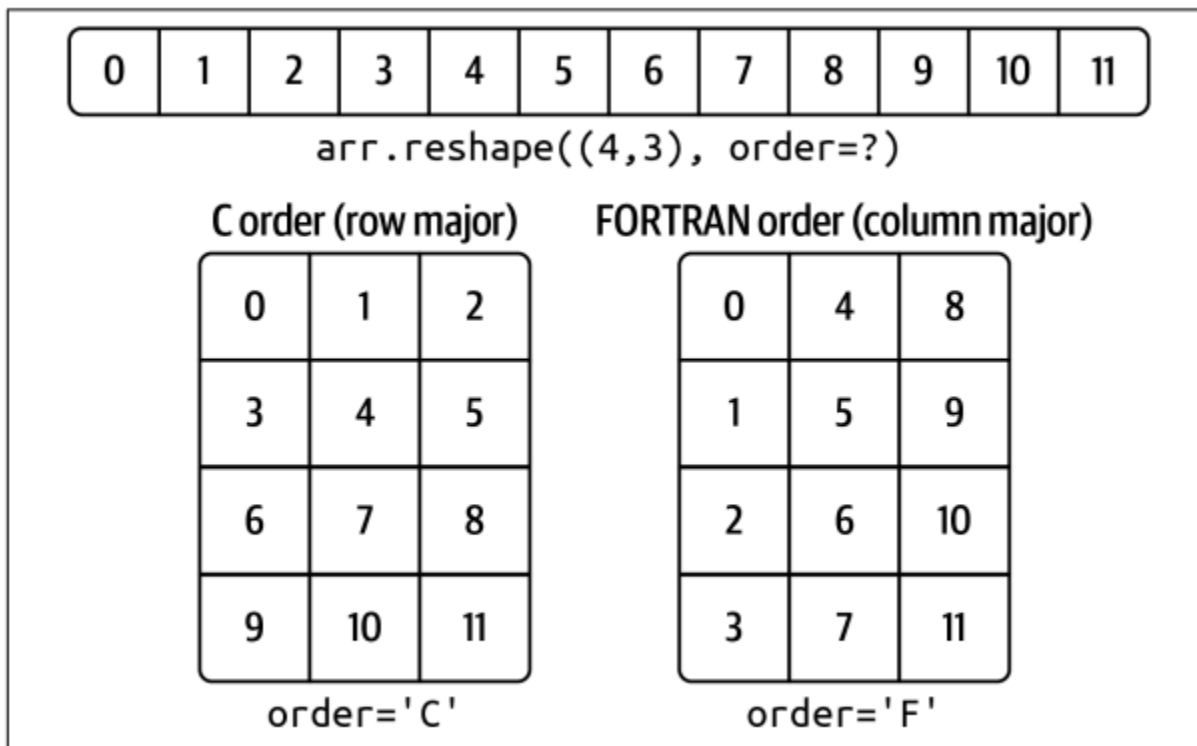


Figure A-3. Reshaping in C (row major) or FORTRAN (column major) order

Mảng đa chiều cũng có thể được định hình lại:

```
arr.reshape((4, 2)).reshape((2, 4))
```

```
array([[0, 1, 2, 3],
       [4, 5, 6, 7]])
```

Một trong các chiều có thể được đặt là -1, trong trường hợp đó, giá trị cho chiều đó sẽ được tự động suy ra từ dữ liệu.

```
arr = np.arange(15)
arr.reshape((5, -1))
```

```
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11],
       [12, 13, 14]])
```

Vì thuộc tính shape của mảng là một tuple nên nó cũng có thể được truyền vào reshape:

```
other_arr = np.ones((3, 5))
other_arr.shape
```

```
(3, 5)
```

```
arr.reshape(other_arr.shape)
```

```
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14]])
```

Thao tác ngược lại của `reshape` từ một chiều sang đa chiều được gọi là ****làm phẳng (flattening)**** hoặc ****đuỗi (raveling)****.

```
arr = np.arange(15).reshape((5, 3))
arr
```

```
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11],
       [12, 13, 14]])
```

```
arr.ravel()
```

```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14])
```

flatten() hoạt động tương tự nhưng luôn trả về một bản sao (copy) của dữ liệu.

```
arr.flatten()
```

```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14])
```

Thứ tự C và FORTRAN (C Versus FORTRAN Order)

Mặc định, mảng NumPy được tạo theo **thứ tự C (row-major)**, nghĩa là các phần tử trong cùng một hàng được lưu trữ liên tiếp trong bộ nhớ. Lựa chọn khác là **thứ tự FORTRAN (column-major)**, nghĩa là các giá trị trong cùng một cột được lưu trữ liên tiếp. reshape và ravel chấp nhận một tham số order ('C' hoặc 'F') để chỉ định thứ tự này.

```
arr = np.arange(12).reshape((3, 4))
arr
```

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

```
arr.ravel()
```

```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

```
arr.ravel('F')
```

```
array([ 0,  4,  8,  1,  5,  9,  2,  6, 10,  3,  7, 11])
```

Nối và Tách mảng (Concatenating and Splitting Arrays)

np.concatenate: Nối một chuỗi các mảng theo một trục cho trước.

np.vstack / np.hstack: Các hàm tiện ích để xếp chồng các mảng theo chiều dọc (hàng) và chiều ngang (cột).

np.split: Tách một mảng thành nhiều mảng con.

```
arr1 = np.array([[1, 2, 3], [4, 5, 6]])
arr2 = np.array([[7, 8, 9], [10, 11, 12]])
np.concatenate([arr1, arr2], axis=0)
```

```
array([[ 1,  2,  3],
       [ 4,  5,  6],
       [ 7,  8,  9],
       [10, 11, 12]])
```

Nối theo cột (axis=1)

```
np.concatenate([arr1, arr2], axis=1)
```

```
array([[ 1,  2,  3,  7,  8,  9],
       [ 4,  5,  6, 10, 11, 12]])
```

Có một số hàm tiện lợi, chẳng hạn như `vstack` và `hstack`, dành cho các kiểu nối thông dụng. Các phép toán trước đó có thể được biểu diễn như sau:

```
np.vstack((arr1, arr2))
```

```
array([[ 1,  2,  3],
       [ 4,  5,  6],
       [ 7,  8,  9],
       [10, 11, 12]])
```

```
np.hstack((arr1, arr2))
```

```
array([[ 1,  2,  3,  7,  8,  9],
       [ 4,  5,  6, 10, 11, 12]])
```

`split`, on the other hand, slices an array into multiple arrays along an axis:

```
arr = rng.standard_normal((5, 2))
arr
```

```
array([[ -1.42382504,  1.26372846],
       [ -0.87066174, -0.25917323],
       [ -0.07534331, -0.74088465],
       [ -1.3677927 ,  0.6488928 ],
       [  0.36105811, -1.95286306]])
```

```
first, second, third = np.split(arr, [1, 3])
first
```

```
array([[ -1.42382504,  1.26372846]])
```

```
second
```

```
array([[ -0.87066174, -0.25917323],
       [ -0.07534331, -0.74088465]])
```

```
third
```

```
array([[ -1.3677927 ,  0.6488928 ],
       [  0.36105811, -1.95286306]])
```

Trình trợ giúp xếp chồng: `r_` và `c_`

Có hai đối tượng đặc biệt trong không gian tên NumPy, `r_` và `c_`, giúp việc xếp chồng các mảng trở nên ngắn gọn hơn:

```
arr = np.arange(6)
arr1 = arr.reshape((3, 2))
arr2 = rng.standard_normal((3, 2))
np.r_[arr1, arr2]
```

```
array([[ 0.          ,  1.          ],
       [ 2.          ,  3.          ],
       [ 4.          ,  5.          ],
       [ 2.34740965,  0.96849691],
       [-0.75938718,  0.90219827],
       [-0.46695317, -0.06068952]])
```

```
np.c_[np.r_[arr1, arr2], arr]
```

```
array([[ 0.          ,  1.          ,  0.          ],
       [ 2.          ,  3.          ,  1.          ],
       [ 4.          ,  5.          ,  2.          ],
       [ 2.34740965,  0.96849691,  3.          ],
       [-0.75938718,  0.90219827,  4.          ],
       [-0.46695317, -0.06068952,  5.          ]])
```

Ngoài ra, chúng có thể dịch các lát cắt thành mảng:

```
np.c_[1:6, -10:-5]
```

```
array([[ 1, -10],
       [ 2, -9],
       [ 3, -8],
       [ 4, -7],
       [ 5, -6]])
```

Xem docstring để biết thêm về những gì bạn có thể làm với `c_` và `r_`.

Các yếu tố lặp lại: xếp gạch và lặp lại

Hai công cụ hữu ích để lặp lại hoặc sao chép các mảng để tạo ra các mảng lớn hơn là các hàm `repeat` và `tile`. `repeat` sao chép từng phần tử trong một mảng nhiều lần, tạo ra một mảng lớn hơn:


```
arr = np.arange(3)
arr
```

```
array([0, 1, 2])
```

```
arr.repeat(3)
```

```
array([0, 0, 0, 1, 1, 1, 2, 2, 2])
```

Theo mặc định, nếu bạn truyền một số nguyên, mỗi phần tử sẽ được lặp lại với số lần tương ứng. Nếu bạn truyền một mảng số nguyên, mỗi phần tử có thể được lặp lại với số lần khác nhau:

```
arr.repeat([2, 3, 4])
```

```
array([0, 0, 1, 1, 1, 2, 2, 2, 2])
```

Mảng đa chiều có thể có các phần tử được lặp lại dọc theo một trục cụ thể:

```
arr = rng.standard_normal((2, 2))
arr
```

```
array([[ -0.07534331, -0.74088465],
       [-1.3677927 ,  0.6488928 ]])
```

```
arr.repeat(2, axis=0)
```

```
array([[ -0.07534331, -0.74088465],
       [ -0.07534331, -0.74088465],
       [-1.3677927 ,  0.6488928 ],
       [-1.3677927 ,  0.6488928 ]])
```

Lưu ý rằng nếu không có trục nào được truyền, mảng sẽ được làm phẳng trước, điều này có thể không phải là điều bạn mong muốn. Tương tự, bạn có thể truyền một mảng số nguyên khi lặp lại một mảng đa chiều để lặp lại một lát cắt nhất định với số lần khác nhau:

```
arr.repeat([2, 3], axis=0)
```

```
array([[ -0.07534331, -0.74088465],  
       [ -0.07534331, -0.74088465],  
       [-1.3677927 ,  0.6488928 ],  
       [-1.3677927 ,  0.6488928 ],  
       [-1.3677927 ,  0.6488928 ]])
```

```
arr.repeat([2, 3], axis=1)
```

```
array([[ -0.07534331, -0.07534331, -0.74088465, -0.74088465, -0.74088465],  
       [-1.3677927 , -1.3677927 ,  0.6488928 ,  0.6488928 ,  0.6488928 ]])
```

Mặt khác, tile là một lời tắt để xếp chồng các bản sao của một mảng dọc theo một trục. Về mặt trực quan, bạn có thể hình dung nó tương tự như việc "xếp gạch":

```
arr
```

```
array([[ -0.07534331, -0.74088465],  
       [-1.3677927 ,  0.6488928 ]])
```

```
np.tile(arr, 2)
```

```
array([[ -0.07534331, -0.74088465, -0.07534331, -0.74088465],  
       [-1.3677927 ,  0.6488928 , -1.3677927 ,  0.6488928 ]])
```

Đối số thứ hai là số ô; với số vô hướng, việc xếp gạch được thực hiện theo từng hàng, thay vì từng cột. Đối số thứ hai của hàm tile có thể là một bộ chỉ ra cách bố trí của "việc xếp gạch":

```
arr
```

```
array([[ -0.07534331, -0.74088465],  
       [-1.3677927 ,  0.6488928 ]])
```

```
np.tile(arr, (2, 1))
```

```
array([[ -0.07534331, -0.74088465],  
       [-1.3677927 ,  0.6488928 ],  
       [ -0.07534331, -0.74088465],  
       [-1.3677927 ,  0.6488928 ]])
```

```
np.tile(arr, (3, 2))
```

```
array([[ -0.07534331, -0.74088465, -0.07534331, -0.74088465],  
       [-1.3677927 ,  0.6488928 , -1.3677927 ,  0.6488928 ],  
       [ -0.07534331, -0.74088465, -0.07534331, -0.74088465],  
       [-1.3677927 ,  0.6488928 , -1.3677927 ,  0.6488928 ],  
       [ -0.07534331, -0.74088465, -0.07534331, -0.74088465],  
       [-1.3677927 ,  0.6488928 , -1.3677927 ,  0.6488928 ]])
```

Tương đương chỉ số ưa thích: lấy và đặt

Như bạn có thể nhớ lại từ Chương 4, một cách để lấy và thiết lập các tập hợp con của mảng là lập chỉ mục phức tạp bằng cách sử dụng các mảng số nguyên:

```
arr = np.arange(10) * 100  
inds = [7, 1, 2, 6]  
arr[inds]
```

```
array([700, 100, 200, 600])
```

Có những phương pháp ndarray thay thế hữu ích trong trường hợp đặc biệt chỉ thực hiện lựa chọn trên một trục duy nhất:

```
arr.take(inds)
```

```
array([700, 100, 200, 600])
```

```
arr.put(inds, 42)
```

```
arr
```

```
array([ 0, 42, 42, 300, 400, 500, 42, 42, 800, 900])
```

```
arr.put(inds, [40, 41, 42, 43])
```

```
arr
```

```
array([ 0, 41, 42, 300, 400, 500, 43, 40, 800, 900])
```

Để sử dụng theo các trục khác, bạn có thể truyền từ khóa axis:

```
inds = [2, 0, 2, 1]
```

```
arr = rng.standard_normal((2, 4))
```

```
arr
```

```
array([[ 0.36105811, -1.95286306,  2.34740965,  0.96849691],  
       [-0.75938718,  0.90219827, -0.46695317, -0.06068952]])
```

```
arr.take(inds, axis=1)
```

```
array([[ 2.34740965,  0.36105811,  2.34740965, -1.95286306],  
       [-0.46695317, -0.75938718, -0.46695317,  0.90219827]])
```

put không chấp nhận đối số trục mà thay vào đó lập chỉ mục vào phiên bản phẳng (một chiều, theo thứ tự C) của mảng. Do đó, khi bạn cần đặt các phần tử

bằng mảng chỉ mục trên các trục khác, tốt nhất nên sử dụng lập chỉ mục dựa trên [].

A.3. Broadcasting

Broadcasting là quy tắc điều chỉnh cách các phép toán hoạt động giữa các mảng có hình dạng khác

nhau.

Quy tắc: Hai mảng tương thích để "broadcast" nếu, đối với mỗi chiều (xét từ cuối), độ dài của các trục hoặc là bằng nhau, hoặc một trong hai bằng 1.

Broadcasting điều chỉnh cách thức hoạt động giữa các mảng có hình dạng khác nhau. Đây có thể là một tính năng mạnh mẽ, nhưng cũng có thể gây nhầm lẫn, ngay cả với người dùng có kinh nghiệm. Ví dụ đơn giản nhất về broadcasting xảy ra khi kết hợp một giá trị vô hướng với một mảng:

```
: arr = np.arange(5)
arr

: array([0, 1, 2, 3, 4])
```

Ở đây, ta nói rằng giá trị vô hướng 4 đã được truyền đến tất cả các phần tử khác trong phép nhân. Ví dụ, ta có thể giảm giá trị trung bình của mỗi cột trong một mảng bằng cách trừ đi giá trị trung bình của cột. Trong trường hợp này, chỉ cần trừ đi một mảng chứa giá trị trung bình của mỗi cột:

```
arr = rng.standard_normal((4, 3))
arr.mean(0)

array([0.1205802 , 0.24301074, 0.14436756])
```

```
demeaned = arr - arr.mean(0)
demeaned
```

```
array([[ 1.60415973,  2.37514869,  0.63299379],
       [ 0.708053  , -1.20199905, -1.35375584],
       [-1.53287221,  0.29853609,  0.60757184],
       [-0.77934052, -1.47168573,  0.11319021]])
```

```
demeaned.mean(0)

array([ 5.55111512e-17, -1.11022302e-16,  0.00000000e+00])
```

Ngay cả với tư cách là một người dùng NumPy có kinh nghiệm, tôi thường thấy mình phải dừng lại và vẽ sơ đồ khi suy nghĩ về quy tắc broadcasting. Hãy xem xét ví dụ cuối cùng và giả sử chúng ta muốn trừ giá trị trung bình khỏi mỗi hàng. Vì `arr.mean(0)` có độ dài là 3, nó tương thích để broadcasting trên trục 0 vì chiều cuối trong `arr` là 3 và do đó khớp. Theo quy tắc, để trừ trên trục 1 (tức là trừ giá trị trung bình của mỗi hàng khỏi mỗi hàng), mảng nhỏ hơn phải có hình dạng (4, 1):

```
arr
```

```
array([[ 1.72473993,  2.61815943,  0.77736134],
       [ 0.8286332 , -0.95898831, -1.20938829],
       [-1.41229201,  0.54154683,  0.7519394 ],
       [-0.65876032, -1.22867499,  0.25755777]])
```

```
row_means = arr.mean(1)
row_means.shape
```

```
(4,)
```

```
row_means.reshape((4, 1))
```

```
array([[ 1.70675357],
       [-0.44658113],
       [-0.03960193],
       [-0.54329251]])
```

```
demeaned = arr - row_means.reshape((4, 1))
demeaned.mean(1)
```

```
array([-1.48029737e-16,  3.70074342e-17,  0.00000000e+00,  3.70074342e-17])
```

Phát sóng qua các trục khác

```
arr - arr.mean(1).reshape((4, 1))
```

```
array([[ 0.01798636,  0.91140586, -0.92939222],
       [ 1.27521433, -0.51240718, -0.76280715],
       [-1.37269008,  0.58114876,  0.79154132],
       [-0.11546781, -0.68538247,  0.80085028]])
```

use the special `np.newaxis` attribute along with “full” slices to insert the new axis:

```
arr = np.zeros((4, 4))
arr_3d = arr[:, np.newaxis, :]
arr_3d.shape
```

```
(4, 1, 4)
```

```
arr_1d = rng.standard_normal(3)
arr_1d[:, np.newaxis]
```

```
array([[ 0.31290292],
       [-0.13081169],
       [ 1.26998312]])
```

```
arr_1d[np.newaxis, :]
```

```
array([[ 0.31290292, -0.13081169,  1.26998312]])
```

Thus, if we had a three-dimensional array and wanted to demean axis 2, we would need to write:

```
arr = rng.standard_normal((3, 4, 5))
depth_means = arr.mean(2)
depth_means
```

```
array([[ 0.04314136,  0.27468984, -0.18852342, -0.20137996],
       [-0.57324159, -0.54671393,  0.11832783, -0.63005577],
       [ 0.09723001,  0.59537117,  0.03307289, -0.6002202 ]])
```

```
depth_means.shape
```

```
(3, 4)
```

```
demeaned = arr - depth_means[:, :, np.newaxis]
demeaned.mean(2)
```

```
array([[ 4.44089210e-17, -1.11022302e-17,  8.88178420e-17,
        -1.66533454e-17],
       [ 2.22044605e-17, -4.44089210e-17, -2.22044605e-17,
        -4.44089210e-17],
       [ 4.44089210e-17,  6.66133815e-17,  0.00000000e+00,
         8.88178420e-17]])
```

Thiết lập giá trị mảng bằng cách phát sóng

The same broadcasting rule governing arithmetic operations also applies to setting values via array indexing. In a simple case, we can do things like:

```
arr = np.zeros((4, 3))
arr[:] = 5
arr
```

```
array([[5., 5., 5.],
       [5., 5., 5.],
       [5., 5., 5.],
       [5., 5., 5.]])
```

However, if we had a one-dimensional array of values we wanted to set into the columns of the array, we can do that as long as the shape is compatible:

```
col = np.array([1.28, -0.42, 0.44, 1.6])
arr[:] = col[:, np.newaxis]
arr
```

```
array([[ 1.28,  1.28,  1.28],
       [-0.42, -0.42, -0.42],
       [ 0.44,  0.44,  0.44],
       [ 1.6 ,  1.6 ,  1.6 ]])
```

```
arr[:2] = [[-1.37], [0.509]]
arr
```

```
array([[-1.37 , -1.37 , -1.37 ],
       [ 0.509,  0.509,  0.509],
       [ 0.44 ,  0.44 ,  0.44 ],
       [ 1.6 ,  1.6 ,  1.6 ]])
```

A.4. Sử dụng ufunc Nâng cao (Advanced ufunc Usage)

Các hàm ufunc nhị phân (binary ufuncs) có các phương thức đặc biệt cho các hoạt động vector hóa.

reduce: Áp dụng một phép toán lặp lại để tổng hợp các giá trị. Ví dụ, `np.add.reduce(arr)` tương đương với `arr.sum()`.

accumulate: Tương tự `reduce` nhưng trả về một mảng chứa các kết quả tích lũy trung gian. `np.add.accumulate(arr)` tương đương `arr.cumsum()`.

outer: Tính toán tích ngoài (cross product) của tất cả các cặp phần tử giữa hai mảng.

reduceat: Thực hiện một phép "reduce" cục bộ trên các lát (slice) của một mảng.

A.5. Mảng có cấu trúc và Mảng bản ghi (Structured and Record Arrays)

ndarray là một container dữ liệu đồng nhất. Tuy nhiên, **mảng có cấu trúc (structured array)** là một ndarray mà mỗi phần tử có thể được coi như một struct trong C, cho phép lưu trữ dữ liệu không đồng nhất.

Bạn có thể định nghĩa một dtype có cấu trúc bằng một danh sách các tuple, mỗi tuple chứa (tên_trường, kiểu_dữ_liệu).

Kiểu dữ liệu lồng nhau và trường đa chiều

When specifying a structured data type, you can additionally pass a shape (as an int or tuple):

```
dtype = [('x', np.int64, 3), ('y', np.int32)]
arr = np.zeros(4, dtype=dtype)
arr

array([([0, 0, 0], 0), ([0, 0, 0], 0), ([0, 0, 0], 0), ([0, 0, 0], 0)],
      dtype=[('x', '<i8', (3,)), ('y', '<i4')])
```

In this case, the x field now refers to an array of length 3 for each record:

```
arr[0]['x']
```

```
array([0, 0, 0])
```

Conveniently, accessing arr['x'] then returns a two-dimensional array instead of a one-dimensional array as in prior examples:

```
arr['x']

array([[0, 0, 0],
       [0, 0, 0],
       [0, 0, 0],
       [0, 0, 0]])
```

This enables you to express more complicated, nested structures as a single block of memory in an array. You can also nest data types to make more complex structures. Here is an example:

```
dtype = [('x', [('a', 'f8'), ('b', 'f4')]), ('y', np.int32)]
data = np.array([(1, 2), 5], dtype=dtype)
data['x']

array([(1., 2.), (3., 4.)], dtype=[('a', '<f8'), ('b', '<f4')])
```

```
In [161]: data['y']
```

```
array([5, 6], dtype=int32)
```

```
In [162]: data['x']['a']
```

```
array([1., 3.])
```

Tại sao nên sử dụng mảng có cấu trúc?

So với Pandas DataFrame, mảng có cấu trúc là một công cụ cấp thấp hơn. Chúng hiệu quả cho việc ghi/đọc dữ liệu ra đĩa dưới dạng các bản ghi nhị phân có độ dài cố định, hoặc truyền dữ liệu qua mạng, vì cấu trúc bộ nhớ của chúng tương ứng với các struct trong C.

A.6. Sắp xếp (More About Sorting)

Sắp xếp tại chỗ (In-place sort): Phương thức `arr.sort()` sẽ sắp xếp mảng ngay tại chỗ mà không tạo ra một bản sao mới.

Sắp xếp và tạo bản sao: Hàm `np.sort(arr)` sẽ tạo và trả về một bản sao đã được sắp xếp của mảng, giữ nguyên mảng gốc.

Sắp xếp gián tiếp (Indirect Sorts):

`argsort()`: Trả về một mảng chứa các **chỉ số (indices)** mà sẽ sắp xếp mảng gốc.

`lexsort()`: Thực hiện sắp xếp theo từ điển (lexicographical sort) trên nhiều mảng khóa (key).

A.7. Viết hàm NumPy nhanh bằng Numba (Writing Fast NumPy Functions with Numba)

Vấn đề: Các hàm tự viết bằng Python thuần túy để xử lý mảng (ví dụ: dùng vòng lặp for) thường rất chậm so với các hàm ufunc được viết sẵn bằng C của NumPy.

Giải pháp: Numba là một dự án mã nguồn mở giúp tạo ra các hàm nhanh cho dữ liệu kiểu NumPy bằng cách sử dụng trình biên dịch LLVM để dịch mã Python thành mã máy đã được biên dịch.

numba.jit:

Đây là chức năng chính của Numba, được sử dụng như một **decorator (@nb.jit)** để biên dịch một hàm Python.

Hàm được biên dịch bằng jit có thể đạt tốc độ tương đương, thậm chí nhanh hơn cả các hàm NumPy được vector hóa.

numba.vectorize:

Decorator này được dùng để tạo ra các hàm ufunc đã được biên dịch, có khả năng hoạt động trên từng phần tử của mảng một cách hiệu quả, tương tự như các ufunc có sẵn

của NumPy.

A.8. Thao tác I/O Nâng cao trên Mảng (Advanced Array Input and Output)

Ngoài `np.save` và `np.load` cơ bản, NumPy cung cấp các phương pháp I/O mạnh mẽ hơn.

Tập được Ánh xạ Bộ nhớ (Memory-Mapped Files)

Khái niệm: `mmap` là một phương pháp làm việc với dữ liệu nhị phân trên đĩa **như thể** nó đang nằm trong bộ nhớ RAM.

Mục đích: Cho phép xử lý các tập dữ liệu cực lớn, không thể tải toàn bộ vào bộ nhớ. `mmap` tạo ra một đối tượng mảng (ndarray-like) mà các thao tác trên đó (như slicing) sẽ đọc hoặc ghi trực tiếp trên file đĩa.

Cách sử dụng: Dùng `np.memmap()` để tạo hoặc mở một tập `mmap`, chỉ định đường dẫn, `dtype`, `mode` (chế độ đọc/ghi), và `shape`.

Lưu ý:

Các thay đổi trên một `mmap` sẽ được đệm (buffered) trong bộ nhớ. Cần gọi phương thức `.flush()` để đảm bảo dữ liệu được ghi xuống đĩa.

Khi `mmap` bị xóa (garbage collected), các thay đổi cũng sẽ được tự động ghi lại.

HDF5 và các định dạng lưu trữ khác

HDF5 (Hierarchical Data Format): Là một định dạng file hiệu quả và có khả năng nén, được thiết kế để lưu trữ các tập dữ liệu lớn và phức tạp.

Thư viện Python: `PyTables` và `h5py` là hai thư viện phổ biến cung cấp giao diện thân thiện với NumPy để làm việc với file HDF5. Định dạng này có thể lưu trữ hàng trăm gigabyte hoặc thậm chí terabyte dữ liệu một cách an toàn.

A.9. Mẹo về Hiệu suất (Performance Tips)

Để đạt được hiệu suất tốt nhất khi sử dụng NumPy, hãy tuân thủ các nguyên tắc sau:

1. **Vector hóa:** Chuyển đổi các vòng lặp Python và logic điều kiện thành các phép toán mảng và chỉ mục Boolean.
2. **Sử dụng Broadcasting:** Tận dụng broadcasting bất cứ khi nào có thể để tránh tạo ra các mảng tạm thời không cần thiết.
3. **Sử dụng "View" thay vì "Copy":** Sử dụng các thao tác slicing (`arr[:]`) để tạo ra các "khung nhìn" (view) thay vì sao chép dữ liệu, giúp tiết kiệm bộ nhớ và thời gian.
4. **Sử dụng các phương thức ufunc:** Tận dụng các phương thức như `reduce`, `accumulate` để thực hiện các phép tính tổng hợp một cách hiệu quả.

Tầm quan trọng của Bộ nhớ liên kề (Contiguous Memory)

Khái niệm: Một mảng được gọi là "liên kề" (contiguous) nếu các phần tử của nó được sắp xếp trong bộ nhớ theo đúng thứ tự xuất hiện của chúng trong mảng (theo thứ tự hàng C hoặc cột FORTRAN).

Ảnh hưởng hiệu suất: Các phép toán trên các khối bộ nhớ liên kề thường nhanh hơn đáng kể do cơ chế đệm (caching) của CPU.

Kiểm tra và Chuyển đổi:

Bạn có thể kiểm tra một mảng có liên kề hay không thông qua thuộc tính `arr.flags`.

Nếu một mảng không có thứ tự bộ nhớ mong muốn, bạn có thể tạo một bản sao với thứ tự đúng bằng cách dùng `.copy(order='C')` hoặc `.copy(order='F')`.

Lưu ý: Các "view" được tạo ra từ các thao tác slicing phức tạp (ví dụ: `arr[:, :50]`) không được đảm bảo là sẽ liên kề.

Tóm lại, phụ lục này mở rộng các kiến thức về NumPy, tập trung vào các kỹ thuật tối ưu hóa hiệu suất thông qua việc hiểu rõ cấu trúc dữ liệu, sử dụng các công cụ nâng cao như Numba, và quản lý bộ nhớ một cách hiệu quả.